

DAMAGE-STAGES — our damage formula against the authority, stage by stage

Version: 7.0.0 — 2026-09-20.

WHAT THIS PAGE IS. One row per stage of the authority's damage pipeline, what this engine does at that stage, and the instrument that re-checks every row on every run. It began as an AUDIT of a chain that was wrong in about a third of its members. **That audit landed.** From 7.0.0 the tables below describe the engine that EXISTS rather than the one that was broken, and the version-by-version record of what was wrong is kept, dated and unedited, under THE DATED RECORD at the foot of the page.

READ THE TABLES AGAINST A SYMBOL, NOT A LINE NUMBER. Every earlier version of this page pinned its "ours" column to line numbers inside one frozen release, and every one of those columns was wrong within days of being written — the file is fifty thousand lines and every fix above a row moves it. The columns now name the FUNCTION and the IDENTIFIER, which move only when somebody renames them.

o. THE AUTHORITY FOR THE SECOND HALF OF THIS CHAIN IS THE CHAMPIONS MOD, AND THIS PAGE CITED MAINLINE FOR IT UNTIL 7.0.0

getDamage is **mainline** — sim/battle-actions.ts, not overridden. Everything from the +2 down to the 16 -bit truncation is **not**: modifyDamage is one of the methods data/mods/champions/scripts.ts replaces, beside canTerastallize, canMegaEvo, spreadMoveHit and hitStepMoveHitLoop. Reading sim/battle-actions.ts for those stages is reading MAINLINE where Champions overrides it, which is the failure CLAUDE.md names by name, and it is what this page did for its whole life.

NO STAGE MOVES BECAUSE OF THIS AND EVERY CITATION BELOW STAGE 4 DOES. The override's ORDER is mainline's, clause for clause; what it adds is the two effectiveness announcements (-supereffective and -resisted, each capped by a Math.min) that give the format its 4x and 0.25x lines. The arithmetic either side of them is character-identical. So this is a correction to the EVIDENCE and not to the finding — and a page whose citations point at a file the format does not run is a page that cannot be checked, which is worse than one that is merely out of date.

```
$ grep -n "modifyDamage(\|canTerastallize(\|spreadMoveHit(\|hitStepMoveHitLoop(" \
    $SHOWDOWN_PATH/data/mods/champions/scripts.ts
180:  canTerastallize(pokemon) {
197:  modifyDamage(baseDamage, pokemon, target, move, suppressMessages) {
315:  spreadMoveHit(targets, pokemon, moveOrMoveName, hitEffect?, isSecondary?, isSelf?) {
428:  hitStepMoveHitLoop(targets: Pokemon[], pokemon: Pokemon, move: ActiveMove) {
```

AND ONE STAGE OF THE MAINLINE FORMULA CANNOT HAPPEN HERE AT ALL. canTerastallize returns null in the mod, so pokemon.terastallized is never set: the Tera base-power floor, the Tera STAB doubling and the whole Stellar branch inside modifyDamage are unreachable in this regulation. They are recorded in the table as CANNOT OCCUR rather than as ABSENT, because those are different claims and only one of them is a debt.

1. THE AUTHORITY'S PIPELINE, ONE ROW PER STAGE

Three ways the authority applies a multiplier, and the difference between them is where every finding on this page has ever lived:

how	what it is
modify(v, m)	sim/battle.ts, Battle#modify — <code>tr((tr(v * tr(m*4096)) + 2047) / 4096)</code> . Fixed point, round-half-up on 4096ths.
chainModify / runEvent	Battle#chainModify folds every handler at a stage into ONE event.modifier, spent ONCE by finalModify -> modify. Two handlers in one chain truncate once, not twice.
plain tr()	a bare truncated multiply. The source labels two of these "not a modifier" on purpose .

tr is Dex#trunc, which is num >>> 0 — not Math.floor chosen for taste.

Authority column: A is mainline sim/battle-actions.ts getDamage; C is Champions data/mods/champions/scripts.ts modifyDamage. **Ours** is engine/medicham2-browser.js, inside dmgRangeOneHit unless another function is named.

#	stage	authority	how	ours	verdict
1	BasePower chain — items, abilities, terrain, Helping Hand, Charge, auras, the move's own onBasePower	A runEvent('BasePower'), then clampIntRange(basePower, 1)	chainModify, spent once	_bpChain, fed by BPCH, spent once by mdChain and clamped by Math.max(1, ...)	SAME STAGE, SAME CHAIN
2	Tera base-power floor	A	assignment	absent	CANNOT OCCUR — canTerastallize returns null in the mod (\$0)
3	stat modifiers ModifyAtk / ModifySpA / ModifyDef / ModifySpD	A runEvent('Modify' + statTable[stat]), attacker then defender	chainModify, spent once each	_aCh / _dCh, fed by ACH / DCH, each spent once by mdChain	SAME STAGE, SAME CHAIN
4	base damage tr(tr(tr(tr(2L/5+2)*bp*A)/D)/50)	A	integer	Math.floor(Math.floor(22*mvBP*A/D)/50)+2	SAME — at level 50, 2L/5+2 is 22, and 22*bp*A is already an integer, so the extra tr is a no-op
5	+2	C baseDamage += 2	addition	folded into the line above	SAME
6	spread x0.75	C, modify	modify	md4096(base, 0.75)	SAME STAGE, SAME ARITHMETIC
6b	Parental Bond's second packet x0.25	C, the else if on the spread branch	modify	md4096(base, perHit.bondMult) in the same slot, reached through hitPlanOf's two-packet plan	SAME STAGE. It used to be one packet at x1.25 several stages later; two packets is what the ability is

#	stage	authority	how	ours	verdict
7	weather <code>WeatherModifyDamage</code>	<code>C , priorityEvent</code>	<code>priorityEvent</code> -> <code>chainModify</code>	<code>md4096</code> on the rain and sun pairs, read through the SUPPRESSED sky	SAME STAGE
8	CRIT — a plain <code>tr(x * 1.5)</code> , NOT a modifier	<code>C</code>	<code>tr()</code>	<code>Math.floor(base*1.5)</code> , gated on <code>_critHere</code>	SAME POSITION ON BOTH ROADS. The certain crit was always here; the ROLLED crit used to be multiplied onto this function's OUTPUT by the battle loop, four stages late. The battle loop no longer multiplies — it asks for the crit price through <code>isCrit</code>
9	the randomizer — also NOT a modifier	<code>C -> sim/battle.ts</code> <code>Battle#randomizer ,</code> <code>tr(tr(d*(100-</code> <code>random(16)))/100)</code>	<code>tr()</code>	<code>Math.floor(base*r/100)</code> inside <code>roll()</code>	SAME POSITION, SAME ARITHMETIC. Different SHAPE, documented in <code>engine/game_differential.js</code> ; the shape is not the position
10	STAB (and <code>ModifySTAB</code> for Adaptability)	<code>C , modify</code>	<code>modify</code>	<code>md4096(d , stab)</code>	SAME STAGE
11	type effectiveness , clamped to <code>-6..6</code> , <code>x2</code> per step up and <code>tr(/2)</code> per step down	<code>C</code>	literal	<code>Math.floor(d*eff)</code>	SAME. <code>floor(d/4) === floor(floor(d/2)/2)</code> for every integer, so the single floor IS the reference
12	burn <code>x0.5</code> , physical, not Guts, not Facade	<code>C , modify</code>	<code>modify</code>	<code>md4096(d , burn)</code> , the Facade exemption keyed on <code>conditionalPower.when === 'userStatused'</code> rather than on a move name	SAME STAGE
13	ModifyDamage chain — the final item and ability chain	<code>C</code> <code>runEvent('ModifyDamage')</code>	<code>chainModify</code> , spent once	<code>mod</code> , fed by <code>MODMUL</code> , spent once by <code>mdChain</code>	SAME STAGE AND GENUINELY A CHAIN
13b	Friend Guard (<code>onAnyModifyDamage</code>)	<code>C</code> , the same chain	<code>chainModify</code> , in the same chain	<code>MODMUL(hit.allyDamageMult)</code> ; it arrives on the <code>hit</code> argument, because the ally is not in this function's parameters	SAME CHAIN. It used to be a second <code>md4096</code> on an already-spent number
14	bypassProtect <code>x0.25</code>	<code>C , modify</code> , after the chain is spent	<code>modify</code>	<code>md4096(dmg , piercesProtect.damageMult)</code> at the hit site, per arrival	SAME — a separate spend is correct here, and the authority quarters each hit rather than the total
15	minimum <code>1</code>	<code>C</code> if (<code>gen !== 5 && !baseDamage</code>) <code>return 1</code>	<code>return 1</code>	<code>roll()</code> returns <code>1</code> when the chain truncates to <code>0</code> and the hit connected (<code>eff > 0</code>)	SAME. Absent until a Parental Bond second packet into a resist was found truncating to <code>0</code> here and paying <code>1</code> there
16	16 -bit truncation	<code>C</code> <code>tr(baseDamage, 16)</code> , which is <code>(n >>> 0) % 65536</code>	<code>tr()</code>	absent	ABSENT AND UNREACHABLE — it needs a single packet above <code>65536</code>

2. EVERY MULTIPLIER WE APPLY, CLASSIFIED BY THE AUTHORITY'S STAGE

Stage read from the handler's own event name through `Dex.forFormat('gen9championsvgc2026regmb')` , **never from memory.** Membership is DERIVED from `data/tags.json` and is deliberately not restated here as a list of counts: the tagger regenerates that artifact against a corpus that grows hourly, and every usage figure this page ever typed into a row had moved by the next gate run.

2a. THE CLASS THAT WAS AT THE WRONG STAGE, AND WHERE EVERY MEMBER SITS TODAY

This was the audit's finding: about a third of the engine's multipliers were spent on the FINAL damage, whatever stage the authority spends them at. Each now sits at the authority's own stage, inside the authority's own chain. **The tag is the subject of every row — no member is matched by name unless the row says so.**

multiplier	authority event	where it is now
the type items — every member of <code>damageMultType</code> , the largest class in the audit	<code>onBasePower</code>	the base-power chain, off the tag
Tough Claws, Sharpness, Mega Launcher, Strong Jaw, Punk Rock (offensive), Iron Fist — <code>boostsMoveClass</code>	<code>onBasePower</code>	the base-power chain, through <code>exact4096</code> , with the move-class flag asked of the move
Technician	<code>onBasePower</code> , at the highest priority in the format	the base-power chain. Its gate reads the RAW base power, which is EQUIVALENT rather than sloppy: the relay is still unspent when its handler runs
Sheer Force — <code>removesOwnSecondaries.powerMult</code>	<code>onBasePower</code>	the base-power chain, only for a move that HAD a secondary to remove
Supreme Overlord — <code>boostsFromFallen</code>	<code>onBasePower</code> , a table of exact <code>4096</code> ths	the base-power chain, off the authority's table rather than the tag's <code>perFallen</code> intent
Helping Hand	the condition's <code>onBasePower</code>	the base-power chain, arriving on the <code>hit</code> argument
Expanding Force / Rising Voltage — <code>terrainScaled</code>	<code>onBasePower</code>	the base-power chain, gated on the GROUNDED subject the authority names per move
Muscle Band / Wise Glasses	<code>onBasePower</code>	the base-power chain. Name-wired , because both are <code>untagged</code> in the artifact; the value is the authority's own pair
Dry Skin — <code>halvesTypeDamage.basePowerMult</code>	<code>onSourceBasePower</code> — the DEFENDER reaching into the attacker's base power	the base-power chain. Same tag as the row below, different field, on purpose
the -ate abilities' power half	<code>onBasePower</code>	the base-power chain. The RETYPE half is decided at the top of the function
Thick Fat / Heatproof / Purifying Salt — <code>halvesTypeDamage.attackerStatMult</code>	<code>onSourceModifyAtk</code> / <code>onSourceModifySpA</code> — the STAT stage	the attacker's stat chain
Water Bubble (the offensive half)	<code>onModifyAtk</code> / <code>onModifySpA</code> — the STAT stage	the attacker's stat chain

multiplier	authority event	where it is now
Sniper — critDamageUp	onModifyDamage	the ModifyDamage chain, gated on <code>_critHere</code> . It used to be folded into the crit's plain multiply, which is neither the right stage nor a modifier at all
Friend Guard	onAnyModifyDamage	the ModifyDamage chain (§4)

THE TWO FAILURE MODES INSIDE "WRONG STAGE", BOTH OF WHICH HAD TO BE FIXED TOGETHER. They stay on the page because they are the reason a wrong stage reads as rounding to every human who looks at it:

1. **The stage itself.** A base power passes through `tr(.../D)` and `tr(.../50)` before it becomes damage; a final multiplier does not. The truncations do not commute.
2. **The chain.** The authority folds every handler at one stage into ONE relay and spends it once. A fix that moves a member to the right stage but keeps one `Math.floor` per member is still wrong the moment two members co-occur — and it passes every single-modifier test anybody would write.

The authority's own arithmetic, for the row that started this page:

```
Kingambit Kowtow Cleave (Dark, physical) into Charizard, flat bodies, top roll, no crit.

bp 85
Black Glasses  modify(85, x1.2)  -- the BasePower chain          = 102
base           tr(tr(tr(22 * 102 * 155)/98)/50) + 2              = 72
randomizer     tr(tr(72*100)/100)                                = 72
STAB           modify(72, x1.5)                                  = 108
type Dark vs Fire/Flying = 1x, ModifyDamage chain empty        -> 108

The same multiplier spent on the FINAL damage instead reaches 109. One stage apart, one point out --
and the TOP roll, the roll every older check pinned, is where that gap is SMALLEST.
```

2b. ABSENT — AND EVERY ABSENCE IS NOW A FACT ABOUT THE REGULATION

The four classes this page once listed as missing from the engine — the field terrains, the auras, Charge, and the whole `damageBoost` family — **are wired**, at the base-power or the stat stage, and the gate re-checks them. What is left absent is absent because the regulation cannot produce it, and every line below is derived rather than recalled:

absent	why
Battery, Power Spot, Steely Spirit (<code>onAllyBasePower</code>)	no legal carrier. They are legal ABILITIES and no legal species in this regulation can have one
Dark Aura, Aura Break (<code>onAnyBasePower</code>)	no legal carrier , same derivation. Fairy Aura has exactly one and IS wired — see below
Punching Glove, the plates, the orbs, Soul Dew	<code>isNonstandard: 'Past'</code> — not in this format. Recorded so nobody wires them
Collision Course, Electro Drift, Brine, Retaliator	<code>isNonstandard: 'Past'</code> . This page filed them as a move-table gap for the owner of <code>build_engine_data.js</code> ; that filing is superseded — they are not moves in this regulation
the Tera floor, the Tera STAB doubling, the Stellar branch	<code>canTerastallize</code> returns <code>null</code> in the mod (§0)
ignoreAccuracy	no legal carrier (§5). <code>ignoreEvasion</code> has carriers, and both of its halves are wired

```
$ SHOWDOWN_PATH=... node -e "
const {Dex}=require(process.env.SHOWDOWN_PATH+'/dist/sim');
const D=Dex.forFormat('gen9championsvgc2026regmb');
const legal=x=>x.exists&&!x.isNonstandard&&x.tier!=='Illegal';
const sp=D.species.all().filter(legal);
const carriers=id=>sp.filter(s=>s.abilities&&Object.values(s.abilities)
    .map(a=>D.abilities.get(a).id).includes(id)).map(s=>s.name);
for(const id of ['battery','powerspot','steelyspirit','darkaura','aurabreak','fairyaura'])
    console.log(id, JSON.stringify(carriers(id)));"

battery      []
powerspot    []
steelyspirit []
darkaura     []
aurabreak    []
fairyaura    ["Floette-Mega"]
```

FAIRY AURA IS A BASE-POWER MULTIPLIER, AND ITS PAIR IS NOT A DECIMAL. The handler is mainline — `data/abilities.ts` , `fairyaura.onAnyBasePower` , under an `onAnyBasePowerPriority` , and `grep fairyaura` returns nothing in the mod:

```
onAnyBasePower(basePower, source, target, move) {
  if (target === source || move.category === 'Status' || move.type !== 'Fairy') return;
  if (!move.auraBooster?.hasAbility('Fairy Aura')) move.auraBooster = this.effectState.target;
  if (move.auraBooster !== this.effectState.target) return;
  return this.chainModify([move.hasAuraBreak ? 3072 : 5448, 4096]);
}
```

So Aura Break does not SUPPRESS the aura, it INVERTS it, at the same stage. The artifact's older scalar truncates one `4096` th low; the engine REFUSES a scalar there, counts `auraMultUnusable` , and leaves the damage alone rather than being quietly wrong. The aura applies to every Fairy move on the field rather than only the holder's, which is why the holder's own sheet count was never the exposure — **and the claim this page carried, that the holder is a Gardevoir forme, is false. That forme's ability is Pixilate; Fairy Aura's one legal carrier is the Floette mega forme, reached by its stone on Floette-Eternal .**

2c. SAME STAGE — CHECKED, AND NOW RE-CHECKED ON EVERY RUN

This list exists so the next session does not re-audit it. Each was measured against the authority rather than read, and each is now a row the stage gate derives on every run.

multiplier	authority event	ours
Life Orb	onModifyDamage	folded into mod through ch4096
Metronome (the item)	onModifyDamage , and a LADDER rather than a constant	MODMUL off the tag's own step table
Multiscale / Shadow Shield, Filter / Solid Rock / Prism Armor, Ice Scales, Punk Rock (defensive)	onSourceModifyDamage — not a stat modifier, which is the natural mis-statement	damageReduce , in the chain
Tinted Lens, Expert Belt, Neuroforce	onModifyDamage	in the chain. Neuroforce is name-wired and has no tag entry — correct today, brittle
the resist berries	onSourceModifyDamage	in the chain, keyed on the type this function actually PRICED rather than on the move's static type
Reflect / Light Screen / Aurora Veil	onAnyModifyDamage	DOUBLES_SCREEN , the authority's doubles constant exactly; a crit ignores them
the four Ruin abilities	onAnyModifyDef / onAnyModifyAtk / onAnyModifySpA / onAnyModifySpD — the STAT stage	the stat chains
Huge Power / Pure Power, Guts, Solar Power, Orichalcum Pulse, Hadron Engine	onModifyAtk / onModifySpA	the attacker's stat chain
Flash Fire's absorbed volatile	the condition's onModifyAtk / onModifySpA — a STAT stage	the attacker's stat chain, paid only into the stat the move actually uses
the damageBoost family	onBasePower , or onModifyAtk / onModifySpA , per member	split across the base-power and stat chains by the event the member carries. The narrowed shape and its membership are PRINTED by the gate on every run
Rivalry — damageByGender	onBasePower , with an ELSE that a one-multiplier tag cannot hold	the base-power chain, three ways off genderOf , which answers a declared gender or none
Wonder Room — swapsDefences	Pokemon#getStat swaps storedStats and then applies the ORIGINAL stat's stage	applied to the defence before the stage multiplier, with the defence KEY deliberately left unrewritten
Adaptability	onModifySTAB	stabBoost
the minimize punish — punishesMinimize	the move flag's damage half	MODMUL , and the never-miss half is in hitChance (§5). The tag names BOTH halves precisely so a consumer cannot land one and skip the other
snow raising an Ice body's defence, sand raising a Rock body's	the weather condition's onModifyDef / onModifySpD	the defender's stat chain — and gated on the STAT the hit reads, not on the move's category

THE WEATHER ROW IS THE ONE THIS TABLE GOT WRONG, AND IT WAS WRONG IN BOTH DIRECTIONS AT ONCE. Which handler a hit runs is move.overrideDefensiveStat || (isPhysical ? 'def' : 'spd'), and overrideDefensiveStat WINS. This format has exactly one such move — Psyshock, special, reading Defence, derived as the only statSwap.attackInto row in data/tags.json — so a category gate missed the boost on a Psyshock into an Ice body in snow and applied one the authority never applies into a Rock body in sand. Every neighbouring multiplier already read the stat; these two lines were the only readers of the category. Knob MEDI_WEATHER_DEF_BY_CATEGORY=1 restores the category gate; probe tests/probe_weather_defence_stat.js .

AND A STAGE ARRIVED AT WRONGLY MAKES EVERY MULTIPLIER ON THIS PAGE CORRECT AND THE DAMAGE WRONG. This chain reads a stat stage; it does not decide how one is arrived at. A charge-turn self-boost was doing raw arithmetic on the boost vector instead of asking the one sign-inverting reader the other boost roads ask, so a swapped inverting ability did not invert it — and a four-stage Special Attack error walks straight through a correct formula. Knob MEDI_CHARGE_BOOST_RAW=1 , probe tests/probe_charge_boost_contrary.js . The same shape is why the stat-drop road, the ability-boost road and this one must share a reader rather than each do the arithmetic.

3. THE TWO NON-MODIFIERS, CHECKED EXPLICITLY

The crit

- Is it a plain truncated x1.5 , or does it go through the 4096 ths helper? Plain, on both engines. That matches the authority's tr(baseDamage * 1.5) and its own "crit - not a modifier" comment.
- Position. The authority puts the crit BEFORE the randomizer, STAB, the type chart, burn and the ModifyDamage chain. Both of this engine's roads are now there. The rolled crit used to multiply a number that had already been rolled, STAB'd, type-charted, burnt and chain-spent — and it hid because every existing check pinned the TOP roll, where the randomizer is the identity.
- The three things a crit IGNORES, from moveHit.crit setting ignoreNegativeOffensive and ignorePositiveDefensive : the attacker's NEGATIVE offensive stages, the defender's POSITIVE defensive stages, and screens. And the one it does not: BURN. Burn's halving is a multiplier, not a stage, so it survives a critical hit. A probe goes RED on any engine that "completes" that list.
- The delayed payout takes the crit through the SAME door. The authority hands a delayed payout to trySpreadMoveHit , so it walks the same step list as a direct click; this engine RE-PRICES it as a certain crit through the damage function rather than multiplying afterwards. A late x1.5 would be wrong in exactly the way the old rolled crit was wrong. The -crit line is written between the effectiveness line and the damage line, which is the authority's order.
- The rate is a rate and stays out of the range. The crit is drawn as randomChance(1, critMult[critRatio]) , and gen 9 clamps the ratio to a stage whose critMult entry is 1 — so a guaranteed crit is a draw that is always true, not a draw that is skipped.

The roll

- Position: SAME, between the crit and STAB, confirmed at both endpoints on every control row.
- Shape: different, and already documented in engine/game_differential.js — uniform integers here against the authority's inverted indices, agreeing at the endpoints. That note is about shape and not about position, and the two get confused every time somebody re-reads it.

4. THE CHAIN, NOT ONLY THE STAGE

Friend Guard is `onAnyModifyDamage` , so it belongs in the **same chain** as Life Orb, the screens, the resist berries and Expert Belt — not beside it. This engine used to spend it as its own `md4096` on the number the damage function had already spent its chain on: two spends against the authority's one.

```
Life Orb x1.3 then Friend Guard x0.75, over base damages 20..300

authority      modify(d, chain(chain(1, 1.3), 0.75))    ONE spend
the old road   modify(modify(d, 1.3), 0.75)            TWO spends
-> 60 of 281 base-damage values disagree (21.4%);   at d=45 the authority says 44 and the old road 43
```

The rule that outlives the fix: a member at the right stage in the wrong CHAIN is invisible to any test that switches on one multiplier at a time. Every stage in the gate carries at least one two-member row for exactly that reason, and those rows are the ones that fail first if a per-member floor is ever re-introduced.

5. THE TO-HIT PIPELINE — THE OTHER STAGED CHAIN ON THIS ROAD

A miss is a damage of zero, so the same stage-and-chain discipline decides it, and this page had never carried the pipeline that does. The authority is `hitStepAccuracy` in mainline `sim/battle-actions.ts` ; Champions does not override it, and the only `ignoreAccuracy` / `ignoreEvasion` text in the mod sits inside `hitStepMoveHitLoop` 's `multiaccuracy` branch, which is the later arrivals of a volley and a different question. Ours is `hitChance` .

#	stage	authority	ours
1	the guarantees, decided before any modifier exists	<code>onAnyAccuracy</code> / <code>onSourceAccuracy</code> returning <code>true</code> , plus the step list's own <code>move.alwaysHit</code> , Poison-type Toxic and self-targeting status clauses	<code>Infinity</code> , returned above the arithmetic: No Guard on either body , Lock-On (attacker-side, naming one defender), a Poison-type's Toxic, and a minimize-punishing move into a minimized body
2	the OHKO class	if <code>(move.ohko)</code> sets the accuracy OUTRIGHT — the <code>ModifyAccuracy</code> event AND both stage adjustments live in the <code>else</code>	a branch that RETURNS, with the accuracy, the gate type and the immunity all read out of the <code>ohko</code> tag. The level term is identically zero at level <code>50</code> , which is a format fact rather than an approximation
3	<code>ModifyAccuracy</code> — Bright Powder, Wide Lens, Zoom Lens, Compound Eyes, Hustle, Sand Veil, Snow Cloak, Wonder Skin, Tangled Feet, Gravity	<code>runEvent('ModifyAccuracy')</code> — one chain, spent once	<code>_accMod</code> , accumulated by <code>accChain</code> over the field row and the attacker/defender walk, applied ONCE by <code>accApplyChain</code> . A row carrying a multiplier and no <code>4096</code> this pair is COUNTED (<code>accModNoChainPair</code>), never silently floated
3b	the defender's ability is read THROUGH Mold Breaker	the <code>breakable</code> guard lives inside <code>runEvent</code> itself, so it governs EVERY event, including this one	asked through <code>suppressedAbility</code> , the same reader the bounce, absorb and status roads use. Membership printed before wiring: three breakable accuracy abilities in the format, all defender-side, so the attacker half of the walk cannot move and the ITEM half must not
4	the accuracy and evasion STAGES	two nested <code>clampIntRange(..., -6, 6)</code> , then <code>trunc(accuracy * (3 + boost) / 3)</code> or <code>trunc(accuracy * 3 / (3 - boost))</code>	combined once, clamped once, looked up once, truncated once with <code>>>> 0</code> , the authority's own <code>tr</code>
5	the draw	<code>randomChance(accuracy, 100)</code> for EVERY accuracy that is not literally <code>true</code>	the same rule, with <code>Infinity</code> as this engine's <code>true</code>

THE STAGES SIT BELOW THE MODIFIER WALK, AND THAT IS THE AUTHORITY'S ORDER RATHER THAN A TIDY-UP. The multiplications commute, so the old order was harmless while nothing truncated; the moment a truncation exists, the ORDER decides where the fraction is thrown away. Compound Eyes on a `90` -accuracy move into `+1` evasion is `trunc(117 * 3/4)` one way round and `trunc(90 * 3/4) * 1.3` the other.

AND THE TWO STAGES ARE ONE NUMBER, NOT TWO MULTIPLICATIONS. This engine used to multiply by the accuracy stage, divide by the evasion stage and never truncate. The second clamp is the whole of the caps arm — `-6` minus `+6` is `-12` and must come back as `-6` — and with one side at zero the two roads part wherever the answer is fractional. At `+1` accuracy into `+2` evasion the DIRECTION reverses: the old arithmetic made the attacker MORE accurate than the authority does. It decides hit or miss, so it is board-material. Knob `MEDI_ACC_EVA_SEPARATE=1` .

`ignoreEvasion` **HAS BOTH HALVES WIRED AND THEY ARE TWO DIFFERENT TAGS.** The ABILITY half zeroes the defender's evasion stage off `ignoresEvasion` ; the MOVE half off `ignoresBoosts.evasion` , which is the field the dex flag actually reaches the artifact as. Wiring one and not the other is how this sat half-done for weeks, with a comment in the source naming the missing half as a separate unfixed defect. `ignoreAccuracy` still has no legal carrier and is still not modelled, and that is stated rather than defaulted. Knob `MEDI_MOVE_EVASION_COUNTED=1` , probe `tests/probe_ignore_evasion_move.js` .

6. WHAT THE INSTRUMENT CANNOT SEE, SAID OUT LOUD

`tests/test-damage-stages.js` calls `battle.actions.moveHit` , one level below `spreadMoveHit` . Three events never fire there:

- `ModifyMove` — so the authority never sets `typeChangerBoosted` , and a Pixilate row would compare two different moves. Pixilate is therefore NOT in the matrix; its stage is argued from the handler source in §2a and its behaviour is probed in `tests/test-mechanics.js` . Sheer Force IS in the matrix, with its flag staged by hand exactly as its own `onModifyMove` would set it.
- `spreadHit` and `willCrit` — both staged by hand, and the staging is stated in the file.
- the hit loop**, so `move.hit` and the multi-hit family are outside THIS instrument. They are not outside the engine's coverage: the per-hit differential `tests/test-engine-diff.js` drives the authority's own volley loop through the Champions override, so a volley is compared as a volley rather than as one packet.

Two further limits, stated because they are the shape of the control failures this project keeps finding:

- The reference reports HP ACTUALLY LOST, so a KO clamps it.** The defender is made unfaintable on both sides by the same multiple; without that, "strong" and "much stronger" print the same number and the two engines agree for a reason that has nothing to do with the arithmetic.
- Both abilities are set explicitly on both sides, always.** An early pass left the defender at its species default and measured an ability against itself — the compare-a-Scarf-against-a-Scarf failure, and it happened here before it was caught elsewhere.

AND THE KNOB IS CHECKED BEFORE THE ANSWER IS. Every scenario runs against its own control first, and the gate FAILS if the authority gives the same answer either way: a row that agrees because neither engine did anything is not evidence. One family needs a crit to make its knob visible at all, and that flag is per-row and explicit rather than defaulted on, because a blanket crit would hide a genuinely dead knob everywhere else.

A WHOLE-GAME BOARD FIGURE IS NOT EVIDENCE ABOUT THIS CHAIN, in either direction. A board parts for a targeting error, a status, a switch, a faint or a stale field just as readily as for a damage roll, and a clean stage table is evidence about the damage chain and about nothing else. Name the artifact and the sample whenever either is quoted.

7. THE MEASUREMENT, AND WHAT IS STILL OPEN

Two instruments stand over this page, and both were re-run for 7.0.0 on the engine in the tree:

```
$ SHOWDOWN_PATH=... node tests/test-damage-stages.js
DAMAGE STAGE GATE – exact equality against the authority, all 16 rolls, both crit states
1696/1696 exact
CH_EXACT: 5 override(s), re-derived from the live dex, 0 wrong
narrowed damageBoost shape matches 5: firemane, blaze, overgrow, swarm, torrent – 0 at the wrong stage
PASS

$ grep -c "row(" tests/test-damage-stages.js          # scenarios x 2 crit states x 16 rolls
53
$ grep "row('" tests/test-damage-stages.js | cut -d'"' -f2 | sort | uniq -c
    18 basePower      9 CHAIN      9 stat      5 terrain
     4 control       3 modifyDamage 2 crit      2 weather      1 spread
```

data/engine-diff.json is the second, and it is a different question rather than a bigger version of the first. On release 0d7b1d9db6d1, against Showdown commit 20ad99ffc9a5a4a4e8fb56ab04ad8e4255b3f2b4, it reads requested 6000, compared 6000, agreed 6000 and disagreed 0 — at the midpoint and at every one of its sixteen arms — with seed 20260804, band_missing 0, skipped_multihit 0 and skipped_ability_multihit 0. **The stage gate is EXACT equality on every roll over a small deliberate matrix; the differential is a BAND over a large random draw, and a band cannot see one point.** Neither replaces the other and neither is evidence about the other's blind spot.

WHAT IS STILL OPEN ON THIS CHAIN, EACH WITH ITS REASON.

- **The 16-bit truncation is absent and unreachable** (stage 16). It needs a single packet above 65536 . Recorded, not ranked.
- **Four members are NAME-WIRED and correct today:** Muscle Band, Wise Glasses, Neuroforce and the field terrain constants. Each is named in the tables above and at its site; a tag that starts carrying one retires the name.
- **A member with no derivable subject is LOUD, not defaulted.** A terrainScaled row whose grounded subject is unknown keeps today's answer and counts terrainScaledSubjectUnknown with the offending id, so a member arriving next regulation announces itself instead of inheriting a guess. The same shape guards the aura pair (auraMultUnusable), the accuracy chain (accModNoChainPair , accModNoTagValue) and the move-table read (accuracyNoTable).
- data/engine-diff.json **names what it still cannot draw** rather than rounding it away: a smart-target volley that the authority's own loop throws on at a one-target entry point, and clicks the authority refuses before their first arrival. Both are conditions of the AUTHORITY at that entry point rather than engine defects, and the whole-game differential is the guard on them.
- **Two files outside docs/ still quote this page's superseded scenario count** — tests/bench-medicham.js and tests/staged_status_counters.js . Reported here and not edited: they are not this document's to change.

Every stage in \$1 and every row in \$2 is re-derived by tests/test-damage-stages.js on every run, and the census carries probes on the same mechanics beside it. Nothing on this page is an assertion a reader has to take on trust: each is either a line in the authority you can read, or a row an instrument re-measures.

THE DATED RECORD — HISTORY, NOT CURRENT STATE

Everything below this line is dated evidence, kept unedited. It is the version-by-version record of what this chain did and of what was wrong with it, and it is NOT a description of the engine. Several of those blocks say the stage table is owed, or stale, or dated evidence — they were right when they were written, and the debt they name was discharged; \$7 above is the current reading. Section references inside them (\$2a , \$3) point at the tables **as they stood then**, which is why the blocks are not rewritten: a dated claim is never edited to agree with today.

6.1.0 - THE VOLLEY LOOP IS DAMAGE-COMPARED FOR THE FIRST TIME, AND NOTHING IN THE DAMAGE CHAIN MOVED. data/engine-diff.json , generated 2026-09-10T17:54:22Z on release 3c2b2f9ac845 against Showdown commit 20ad99ffc9a5a4a4e8fb56ab04ad8e4255b3f2b4 , reads requested **6,000**, compared **6,000**, agreed **6,000** and disagreed 0 at the midpoint and at every arm in arms — top , bottom and idx01 to idx14 — with seed 20260804 and band_missing 0. skipped_multihit **and** skipped_ability_multihit **both read 0**, and volley.move_rows **130** plus volley.bond_rows **12** say that **142** of those 6,000 comparisons ran as VOLLEYS through the authority own hit loop rather than as one packet. No stage, no multiplier and no rounding on this page changes: the reference entry point moved from moveHit to hitStepMoveHitLoop for a volley row and every single-hit row still enters at moveHit and draws byte-identical output. What remains uncomparred is named in the same object rather than rounded away — volley.unstageable **1** (Dragon Darts, where the authority own loop throws at a one-target entry point) and volley.no_arrival **4** (Sucker Punch and Last Resort, refused by the authority before their first arrival). Beside it accuracy_conformance reads 500 compared and 0 disagreed, accuracy_modifier_conformance 13 handlers over 14 rows with 0 disagreed, and substitute_bypass_conformance 500 compared with missing and extra both empty.

6.0.0 - NOTHING IN THE DAMAGE CHAIN MOVED, AND THE DIFFERENTIAL THAT SAYS SO IS RE-RUN ON THE RELEASE THAT OPENED THE MEDICHAM GATE. data/engine-diff.json , generated 2026-09-10T09:53:03Z on release cbd510bc2b13 against Showdown commit 20ad99ffc9a5a4a4e8fb56ab04ad8e4255b3f2b4 , reads requested **6,000**, compared **6,000**, agreed **6,000** and disagreed 0, at the midpoint and at every arm in arms — top , bottom and idx01 to idx14 — with seed 20260804 and band_missing 0. No stage, no multiplier and no rounding on this page changes. Beside it accuracy_conformance reads 500 compared and 0 disagreed, accuracy_modifier_conformance 13 handlers over 14 rows with 0 disagreed, and substitute_bypass_conformance 500 compared with missing and extra both empty.

THE SKIP WAS A FAMILY, AND THE TWO COUNTS THIS PARAGRAPH CARRIED ARE RETRACTED AT 6.1.0. The volley loop is wired into tests/test-engine-diff.js and both skipped_multihit and skipped_ability_multihit now read 0; the superseded figures are recorded in docs/RUNNING-NOTES.md and CHANGELOG 6.1.0, and the 6.1.0 block at the top of this page carries the replacement. pool.dropped 9 prior rows still have no damage-table row and can never be drawn.

AND THE HABIT THAT MAKES THIS RE-RUN LOOK RED IS THE SAME ONE NAMED AT 5.265.0. `tests/test-engine-diff.js` invoked with `--n` defaults to 150 comparisons against a published 6,000-comparison artifact. A smaller sample is not a contradiction of the larger one, and reading it as a regression is how a green chain gets reported red.

5.266.0 - NOTHING IN THE DAMAGE CHAIN MOVED, AND THIS TIME THE DIFFERENTIAL THAT SAYS SO IS RE-RUN AND PUBLISHED RATHER THAN WITHHELD. 5.265.0 withheld it: the hazard-sweep order fix had moved `engine/medicham2-browser.js`, so `data/engine-diff.json` had been measured on release `d9e551ed0d5a` while the tree was `57679ef9a4a3`, and a damage verdict about other bytes is not a damage verdict. The re-run is done on the current release. `data/engine-diff.json` **on release 57679ef9a4a3 reads 0 disagreements of 6,000 comparisons**, seed 20260804, requested 6,000, clean at the midpoint and at every one of the sixteen band indices. No file in the damage chain was edited on this pass and no release SOURCE was touched.

THE SKIP IS A FAMILY AND IT IS STATED EVERY TIME, BECAUSE IT IS NOT A ROUNDING ERROR. 134 comparisons were not made. They are the multi-hit moves: 14 of the 500 legal moves carry the `multiHit` tag and are skipped by construction, because one `moveHit` call is one packet. Eleven were drawn and skipped; three were never drawn at all. **The volley loop has still never been damage-compared**, and a clean 0 of 6,000 says nothing about it.

AND THE HABIT THAT MAKES THIS RE-RUN LOOK RED IS THE SAME ONE NAMED AT 5.265.0. `tests/test-engine-diff.js` invoked with `--n` defaults to 150 comparisons against a published 6,000-comparison artifact. A smaller sample is not a contradiction of the larger one, and reading it as a regression is how a green chain gets reported red.

5.265.0 - NOTHING IN THE DAMAGE CHAIN MOVED, AND THIS TIME THE DIFFERENTIAL THAT WOULD SAY SO IS WITHHELD RATHER THAN QUOTED. The engine change in this version is an ORDERING one — `sweepField` ran one clause order for all three carriers of the hazard sweep, and the authority uses three. It touches no stage, no multiplier and no rounding on this page. But it did move `engine/medicham2-browser.js`, so the tree is release `57679ef9a4a3` while `data/engine-diff.json` was measured on `d9e551ed0d5a`, and `engine/status.js` calls that clause MEASURED AGAINST A DIFFERENT ENGINE. **Every count in that artifact is withheld and none is repeated on this page.** That is deliberate and it is the harder of the two available sentences: the damage chain almost certainly did not move, and "almost certainly" is not a measurement. The re-run is `SHOWDOWN_PATH=... node tests/test-engine-diff.js --n 6000 --seed 20260804`.

AND THE HABIT THAT MAKES THAT RE-RUN LOOK RED IS NAMED HERE ONCE. `tests/test-engine-diff.js` invoked with `--n` defaults to 150 comparisons against a published 6,000-comparison artifact, so `engine/publish_guard.js` refuses the shrink and sets a non-zero exit. **No engine state can make that invocation green.** An exit code from a run at the default `--n` is a statement about the publish guard and not about this document's stage table.

5.264.0 - NOTHING IN THE DAMAGE CHAIN MOVED, AND FOR ONCE THAT IS TRUE BY CONSTRUCTION RATHER THAN BY MEASUREMENT. The only file changed this version is `engine/game_differential.js`, the measuring instrument. No engine source moved, so the engine release did not move either: `data/engine-diff.json` on release `d9e551ed0d5a` is the SAME run this page cited at 5.263.0, and it reads **0 of 6000** disagreements at the midpoint and **0 of 6000** at each of the sixteen band indices, seed 20260804, with 134 multi-hit comparisons skipped by construction. No stage, no multiplier and no ordering in this document changes.

THE ONE THING WORTH CARRYING ONTO THIS PAGE IS ABOUT DICE, NOT DAMAGE. The defect corrected this version is that `BattleActions#selfDrops` draws a `random(100)` whose value no legal move in this format ever reads, and that dead draw was shifting the ADDRESS of every later draw in the same any bucket. **The damage roll is not in that bucket** — it is drawn inside `getDamage` and carries its own pinned category, which is why the sixteen band indices above are unaffected and were re-read rather than assumed. The header moves because the project version moves.

5.263.0 - NOTHING IN THE DAMAGE CHAIN MOVED, AND THE DIFFERENTIAL THAT WOULD SAY SO WAS RE-RUN RATHER THAN ASSUMED. `data/engine-diff.json` on release `d9e551ed0d5a` reads **0 of 6000** disagreements at the midpoint and **0 of 6000** at each of the sixteen band indices, seed 20260804, with 134 multi-hit comparisons skipped by construction. No stage, no multiplier and no ordering in this document changes.

ONE OF THE FIVE FIXES TOUCHES A RESIDUAL AMOUNT AND IS WORTH NAMING FOR THAT REASON. A third tag, `scalesOwnStatusDamage`, now sits beside `refusesIndirectDamage` and `healsFromOwnStatus`: Magic Guard refuses a class of damage, Poison Heal converts one status into a heal, and Heatproof MULTIPLIES its own burn by one half. **This is not the damage formula on this page.** The burn chip is a residual, computed from max HP, and it is clamped to a minimum of 1 on both sides of the `Damage` event — which is why a 146-HP body takes 9 and then 4 rather than a single halved figure. The stage table below describes the move-damage chain and is untouched by it.

THE OTHER FOUR ARE TIMING AND ADDRESSING AND TOUCH NO MULTIPLIER. `Battle#boost` refusing on an empty foe side; a damaging pivot that emptied a side not pivoting; the mega phase running below the previous action's `Update` pass; and a fainted body's ability no longer being ignored. The header moves because the project version moves.

5.262.0 - NOTHING IN THE DAMAGE CHAIN MOVED, AND THE DIFFERENTIAL THAT WOULD SAY SO WAS RE-RUN RATHER THAN ASSUMED. `data/engine-diff.json` on release `14b62cd5aeec` reads **0 of 6000** disagreements at the midpoint and **0 of 6000** at each of the sixteen band indices, seed 20260804, with 134 multi-hit comparisons skipped by construction. No stage, no multiplier and no ordering in this document changes.

ONE OF THE SIX SITS ADJACENT TO THIS CHAIN AND IS WORTH NAMING FOR THAT REASON. King's Shield's stat punish now goes through `Battle#boost` rather than writing the vector into `boosts`. **The Attack STAGE that reaches the damage formula is the same number it always was when nothing interferes** — what changed is that three abilities are allowed to interfere with it: Contrary inverts the sign, the Defiant class retaliates, and the Clear Body class refuses the drop outright. The stage table on this page describes what the formula does with a stage; it does not describe how a stage is arrived at, and this fix is entirely on the second half. It is the FACTS-ARE-GLOBAL rule landing one site late: twelve other stat-drop sites in the same file already went through `applyStatDrop`.

THE OTHER FIVE ARE TIMING AND ADDRESSING AND TOUCH NO MULTIPLIER. An ability arriving mid-battle running its own `Start`; the authority's second in-move `eachEvent('Update')` below the recoil; Pickpocket paid on `AfterMoveSecondary`; the status road settling before a Shed Tail departure; and a redirect clearing `move.smartTarget`. The header moves because the project version moves.

5.260.0 - NOTHING IN THE DAMAGE CHAIN MOVED, AND THE DIFFERENTIAL THAT WOULD SAY SO WAS RE-RUN RATHER THAN ASSUMED. All four items in this version are ANNOUNCEMENT and REFUSAL-ORDER defects: a missing `-activate` above a move line, an item announcement sitting above the hit steps instead of inside them, `mustrecharge`'s position in the `BeforeMove` chain, and a missing `-fail` for a move with nothing to aim at. No stage, no multiplier and no truncation position in this document is affected. `tests/test-engine-diff.js --n 6000 --seed 20260804` was re-run on the settled release `a985300cb8ed` and reads **0 disagreements at the midpoint and 0 at every one of the sixteen roll indices**, which is the reading this page exists to carry.

ONE OF THE FOUR SITS ADJACENT TO THIS CHAIN AND IS WORTH NAMING FOR THAT REASON. The item announcement moved into the per-target STEP LIST, beside the step that breaks screens — both are a move's own `onTryHit`, which the authority fires from inside `spreadMoveHit` at the top of the hit loop. The step list is the structure this page's ordering claims rest on, and the change adds an entry to it without moving any existing one.

5.259.0 - NOTHING IN THE DAMAGE CHAIN MOVED. This version is a documentation correction to two published figures elsewhere — a mis-cited quality-filter statistic and a quarantined leaf calibration figure. No stage, no multiplier and no ordering in this document is affected, and the header moves only because the project version does.

5.258.0 - NO DAMAGE STAGE MOVED AND NO MULTIPLIER VALUE CHANGED, BUT TWO OF TONIGHT'S FIVE ITEMS SIT CLOSE ENOUGH TO THIS CHAIN THAT SAYING SO IS THE POINT OF THE BLOCK. Nothing in this document's stage table, its multiplier classes or its ordering changes. **FAIRY AURA IS A BASE-POWER MODIFIER AND ITS MULTIPLIER WAS ALWAYS CORRECT** - what was wrong was the FIELD it read. `field.aura` had two writers where the identically-shaped `recomputeWeatherSuppression` has four, so a holder that had left, returned or been killed went on pricing Fairy moves at a value that was itself right. **A stage table cannot see that defect and would have called this engine clean**, because the stage is asked *what does this multiplier do* and the defect was *is this multiplier present*. **BIG ROOT IS AN ORDERING FACT ON THE HEAL CHAIN, NOT THE DAMAGE CHAIN**, and it is the same shape one road over: `Battle#heal` truncates the base value BEFORE it runs `TryHeal`, so `trunc(155/16) = 9` then `modify(9, 5324, 4896) = 12`, while folding the multiplier into the fraction reaches 12 by luck and truncating after a float multiply reaches 11. **Truncation position is the thing this page exists to pin, and it was wrong on a chain this page does not cover.**

AND THE DAMAGE ENDPOINTS WERE THE VISIBLE HALF OF TONIGHT'S INSTRUMENT DEFECT, WHICH IS A CLAIM ABOUT THE HARNESS AND NOT ABOUT THE FORMULA. Two of the four failures of `tests/test-game-differential.js` were its damage-endpoint clauses, and all four were one line: `const PRIMARY_ARM = ARMS[0]` bound three module-scope staged pins to whatever sat first in the arm list, and `ARMS[0]` stopped being the max-damage arm on 2026-08-13 when the opt-in `middle` arm was prepended. **The engine was never implicated and that was measured rather than asserted:** a 14-call control held this engine's damage interior constant at `108..127` while the authority's wandered and on one run lost its own minimum, and once the pins were bound BY NAME the authority's span came DOWN onto this engine's - knock-off `108..177` to `108..127`, contact punish `66..104` to `66..78` - with this engine's own numbers never moving in either scenario. **The two damage tables are exactly equal on all sixteen rolls whenever the harness stops critting on one side only.** A damage clause that was green for fourteen days because a hash happened to land favourably is not evidence about the formula, and this page should be the last to read it as such.

THE STAGE-BY-STAGE INSTRUMENT WAS NOT RE-RUN IN THIS PASS AND IS STILL OWED, AND THE ENGINE MOVED AGAIN UNDER IT. `engine/medicham2-browser.js` was edited twice tonight, for Fairy Aura and for Beat Up's ally order. Neither touches a multiplier value and neither is on this chain, and neither of those facts discharges the debt: **a stage table is a PINNED measurement, the release it was pinned to is superseded again, and the correct response is a re-run rather than an argument that the fixes cannot have reached a multiplier.** The stage artifact also still predates the instrument digest introduced one version ago, so it cannot say which bytes of the harness produced it. **An unstamped artifact is a figure to WITHHOLD and re-measure, never one to argue about.**

WHICH ARTIFACT HOLDS WHICH WHOLE-GAME FIGURE, AND IT IS NOT EVIDENCE ABOUT THE DAMAGE CHAIN. `data/game-differential.json` was republished off a settled tree this pass and holds board-material 50 of 961 with protocol first-divergence 151 of 961, on release `db248fe67a5e` at cap 20 with the empirical arm and the frozen pool. There is no longer a correctly-measured value of that quantity sitting outside the published artifact. **A whole-game count says nothing about the formula:** a board parts for a targeting error, a status, a switch, a faint or a stale field just as readily as for a damage roll, and a clean stage table is evidence about the damage chain and about nothing else. The damage differential itself is unchanged at **0 of 6000** disagreements at the midpoint and at all sixteen corners, seed 20260804.

5.257.0 - NO DAMAGE STAGE MOVED, NO MULTIPLIER CHANGED, AND NO DAMAGE VALUE WAS RE-DERIVED IN THIS PASS. Nothing in this document's stage table, its multiplier classes or its ordering changes. This version touched a DRIVER rule, the comparator's leaf set and the instrument's own digest - the stand-in player's candidate list, which fourteen more pieces of hidden state are compared, and whether a run can say which bytes measured it. **A LEAF IS STATE, NOT A MULTIPLIER**, so widening the comparison from 40 leaves to 54 adds nothing to this chain and removes nothing from it. The one figure on this page that a reader might be tempted to attach to the damage chain - board-material 34 to 47 on the paired empirical run - is a count of games whose BOARDS parted, and a board parts for a targeting error, a status, a switch or a faint just as readily as for a damage roll.

THE STAGE-BY-STAGE INSTRUMENT WAS NOT RE-RUN IN THIS PASS, AND IT IS STILL OWED FROM THE VERSION BELOW. This version's changelog records no run of the stage-by-stage comparison, so the last measured stage result remains the one recorded in the 5.254.0 block below, on the release named there. It is carried forward and is not restated here as a fresh measurement. The 5.256.0 block below already recorded that this measurement had gone from merely old to OWED, because `medicham2-browser.js` - the file this page's instrument compares against the authority - moved under it. Nothing in this version discharges that. **A stage table is a PINNED measurement and the release it was pinned to is superseded; the correct response is a re-run, not an argument that a driver fix cannot have touched a multiplier.**

AND THIS VERSION ADDS A REASON TO RE-RUN IT THAT THIS PAGE DID NOT PREVIOUSLY HAVE: THE INSTRUMENT IS NOW DIGESTED, AND THE STAGE ARTIFACT PREDATES THAT STAMP. A measurement here pins the engine release, the census and the team pool. All three are INPUTS. Until this version nothing recorded the bytes of the code that READS them, and `engine/arms_comparable.js` was measured answering "COMPARABLE" about two runs that were playing different driver code. `engine/steering.js` now digests the instrument's require closure into `steering.driver_code` and `engine/game_differential.js` voids a run whose instrument moved while it played. The stage-by-stage artifact carries no such stamp, because it was written before the stamp existed - so it cannot say which instrument produced it. That is a reason to re-take the measurement, and it is not a reason to doubt the stage table itself: **an unstamped artifact is a figure to WITHHOLD and re-measure, never one to argue about.**

WHICH ARTIFACT HOLDS WHICH WHOLE-GAME FIGURE, WITH THE ARM NAMED, AND NEITHER IS EVIDENCE ABOUT THE DAMAGE CHAIN. `data/game-differential.json` was again NOT republished and still holds board-material 46 of 961; that is the figure the gate prints and it is stale as a description of tonight's engine. Tonight's runs live in the verification artifacts under `data/verification/`, one per arm: the empirical arm reads 147 protocol and 55 board-material on the repaired driver, and the joint arm's 53 stands and repeats. **Two corrections carried onto this page for completeness, because both were published tonight and both touch how a figure on any page is read:** the joint arm was never non-deterministic - six runs split cleanly either side of the driver fix and each group is bit-identical inside itself - and the empirical figure of 47 is the pre-widening comparator's reading of the same run that the widened comparator reads as 55.

5.256.0 - NO DAMAGE STAGE MOVED, NO MULTIPLIER CHANGED, AND NO DAMAGE VALUE WAS RE-DERIVED IN THIS PASS. Nothing in this document's stage table, its multiplier classes or its ordering changes. Two defects landed in the simulator and neither is on this chain. **WHICH BODY A HIT LANDS ON IS NOT THE DAMAGE CHAIN, AND THE DISTINCTION IS WORTH STATING ON THIS PAGE RATHER THAN ASSUMED.** The headline defect is a targeting error: the release turn of a two-turn move aimed at the lowest live enemy index instead of replaying the aim stored when the move was charged, so a Phantom Force charged at slot b struck slot a. Every multiplier on this page then applied correctly - to the wrong body. This chain computes what a hit does; it does not decide where the hit goes, and a page that only checks the chain would call that engine clean. The second defect is a charge wrapper surviving a BeforeMove refusal, which is a legality question and not a damage one, and it fires 2 times in 961 games.

THE STAGE-BY-STAGE INSTRUMENT WAS NOT RE-RUN IN THIS PASS, AND THE SIMULATOR MOVED UNDER IT. This version's changelog records no run of the stage-by-stage comparison, so the last measured stage result is the one recorded in the 5.254.0 block below, on the release named there. It is carried forward and is not restated here as a fresh measurement. **And it is now owed again rather than merely old:** `engine/engine_release.js` `drift` reports that `medicham2-browser.js` - the file this page's instrument compares against the authority - is the one source that moved between this version's releases. A stage table is a PINNED measurement, and the release it was pinned to is superseded. The correct response to that is a re-run, not an argument that a targeting fix cannot have touched a multiplier.

WHICH ARTIFACT HOLDS WHICH WHOLE-GAME FIGURE, WITH THE ARM NAMED, AND NEITHER IS EVIDENCE ABOUT THE DAMAGE CHAIN. `data/game-differential.json` was again NOT republished and still holds board-material 46 of 961; that published figure is stale as a description of tonight's engine and is what the gate prints. The measurements taken on tonight's engine live in this version's verification artifacts under `data/verification/`, one per driver arm, and they may not be paired with each other: the joint arm reads board-material 53 of 961 with protocol first-divergence 138, and the empirical arm reads 34 with protocol 121. Name the artifact and the arm whenever any of them is quoted. A whole-game figure says nothing about the formula, and a clean stage table is evidence about the damage chain and about nothing else.

5.255.0 - NO DAMAGE STAGE MOVED, NO MULTIPLIER CHANGED, AND NO DAMAGE VALUE WAS RE-DERIVED IN THIS PASS. Nothing in this document's stage table, its multiplier classes or its ordering changes. Two mechanics landed in the simulator and neither is on this chain: Imprison is a legality seal that decides whether a move is played at all, and a bounced Parting Shot is a pivot and a stat drop. Three more comparator leaves were wired and a leaf is state, not a multiplier. **This version's changelog records no re-run of the stage-by-stage instrument, so the last measured stage result is the one recorded in the 5.254.0 block below, on the release named there.** It is carried forward and is not restated as a fresh measurement here; a release cut after it stales it by construction, and the paragraph below is about exactly that.

A RELEASE DIGEST THAT MOVES NOW SAYS WHY, AND THIS PAGE IS ONE OF THE PLACES THAT COSTS THE MOST. The stage table on this page is a PINNED measurement: it is quoted against a frozen release, and a new release id makes it owed again. That is correct when the sources really changed and expensive when they did not. In this version a line-ending change to one source moved a release digest while all 26 frozen sources stayed content-identical, and five heavy clauses were re-run before anybody knew that. `engine/pin_guard.js` now reports each drift as CONTENT-CHANGED or EOL-ONLY, so the next reader of this page can tell a stage table that is genuinely stale from one that only looks stale. **Nothing is excused by this.** The digest still hashes raw bytes, the id still moves, and a stage figure measured against a superseded release is still owed a re-run. The two failing clauses of this version are diagnosed CONTENT-CHANGED and still fail, with their counts withheld.

WHICH ARTIFACT HOLDS WHICH WHOLE-GAME FIGURE, AND NEITHER IS EVIDENCE ABOUT THE DAMAGE CHAIN. `data/game-differential.json` was again NOT republished and still holds board-material 46 of 961. That published figure is stale as a description of tonight's engine; the measurement taken on tonight's engine is board-material 35 of 961 with protocol first-divergence 120, and it lives in this version's verification artifact under `data/verification/`. Name the artifact whenever either is quoted. A whole-game figure says nothing about the formula: a clean stage table is evidence about the damage chain and about nothing else.

THE DAMAGE ROWS IN THE REMAINING WHOLE-GAME LEAVES ARE FENCED, AND THAT IS A STATEMENT ABOUT THE POPULATION AND NOT ABOUT THE FORMULA. Of the 37 whole-game rows standing at the start of this version, roughly 12 are damage-VALUE rows already fenced by a filed row — two engines drawing a different number out of the same legal damage range, which the stage instrument is deliberately built not to compare, because it compares the CHAIN and not the roll. Two more are Poison Touch and are the same shape: a die value, with the ability wired and both engines reaching the draw at the same point. **A die that lands differently is not a formula that computes differently, and this page has no business claiming either way about it.**

5.254.0 - NO DAMAGE STAGE MOVED AND NO MULTIPLIER CHANGED, AND THE STAGE-BY-STAGE INSTRUMENT WAS RE-RUN AFTER EVERY RELEASE THIS VERSION CUT: `data/engine-diff.json` **READS 6,000 COMPARED, 0 DISAGREED, ON RELEASE** `ae608567e8a8`. Nothing in this document's stage table, its multiplier classes or its ordering changes. Four mechanics landed in the simulator and none of them is on this chain: self-inflicted stat drops through a broken Substitute are stat STAGES rather than damage, a sleep attribution is narration, and the sun refusing a freeze is a status immunity. The scope caveat is unchanged and is stated rather than dropped - 134 comparisons skipped as multi-hit, 0 non-finite. **So the stage table below is a CURRENT measurement, re-derived on the engine as it stands after the last release of this version.**

THE SUN FIX BELONGS ON THIS PAGE FOR ONE REASON, AND IT IS A LESSON ABOUT WHAT A WEATHER OBJECT IS. This document treats the sky as a damage multiplier, because that is the only part of it the damage chain touches. The authority's sunnyday condition also carries an `onImmunity` handler that refuses `frz`, and this engine had read only the condition's damage handlers - so a body under sun could be frozen here and not in the real game. **A weather is not a multiplier; it is an object with handlers, and reading the ones this page cares about is not reading the object.** The refusal is read through effective weather rather than off the field, so an ability that suppresses weather puts the freeze back in both engines - asserted as a control.

WHICH ARTIFACT HOLDS WHICH WHOLE-GAME FIGURE, AND THE TWO-NUMBER PROBLEM IS BACK BY DESIGN. `data/game-differential.json` was NOT republished this pass and still holds board-material 46 of 961; the measurement on tonight's engine is 37 of 961 with protocol 122, and it lives in `data/verification/fix-batch-7.json`. The published figure is stale as a description of the engine and is superseded by the verification artifact until a settled-tree pass rewrites it. Name the artifact whenever either figure is quoted. Neither number is evidence about the damage chain: a clean stage table is evidence about the formula and about nothing else.

5.253.0 - NO DAMAGE STAGE MOVED, NO MULTIPLIER CHANGED, AND NO DAMAGE VALUE WAS RE-MEASURED. THIS RELEASE IS A PARSE REPAIR IN THE DEFECT REGISTER. Nothing in this document's stage table, its multiplier classes or its ordering changes. No mechanic was altered in the simulator, no differential ran and no game was played. The stage-by-stage instrument was NOT re-run in this pass; its result is carried forward from the previous version, where it was measured on the current engine.

WHY A REGISTER REPAIR APPEARS ON A DAMAGE PAGE AT ALL. A row whose status cell begins `open - engine DEFECT` was reporting CLOSED to the gate, because the status reader takes the text after the LAST pipe in the row and a code span inside the cell carried a pipe. Any row on this page's subject could have been hidden the same way, in either direction, and eighteen rows were repaired across the register. **None of the eighteen is a damage-stage row, and no stage, multiplier or roll index is touched by any of them.** The published whole-game figure in `data/game-differential.json` is unchanged at board-material 46 of 961.

5.252.0 - NO DAMAGE STAGE MOVED AND NO MULTIPLIER CHANGED, AND THE CURRENCY THIS PAGE HAS OWED SINCE 5.246.0 IS DISCHARGED: THE STAGE-BY-STAGE INSTRUMENT WAS RE-RUN ON THE CURRENT ENGINE. `tests/test-engine-diff.js` ran on release `0dec37ff5ad9` and rewrote `data/engine-diff.json`, which now carries the release id, its cut time, the Showdown commit and a full set of source digests. The measured result is unmoved: 6,000 comparisons requested, 6,000 compared, 6,000 agreed, **0 disagreed**, at the midpoint and at all 16 corner arms - 17 damage indices, seed 20260804. The scope caveat is unchanged and is stated rather than dropped: `skipped_multihit` 134, `skipped_ability_multihit` 17, 0 non-finite. **So the stage table below is a CURRENT measurement again, not dated evidence**, and the six versions of "the currency is still owed" above it are answered by the instrument rather than by an argument.

WHICH ARTIFACT HOLDS WHICH WHOLE-GAME FIGURE - AND THE TWO-NUMBER PROBLEM IS OVER. `data/game-differential.json` was republished this pass and holds board-material 46 of 961 with protocol first-divergence 141. There is no longer a measured figure sitting outside the published one: the sentence this page has carried for five versions, that the gate clause still reads 77 of 961 while the current measurement sits in a `data/verification/` artifact, is **DISCHARGED**. Name the artifact whenever either figure is quoted, exactly as before; what has changed is that there is now only one artifact to name.

WHY A CLEAN STAGE TABLE STILL DOES NOT CLEAR THE GATE, RESTATED BECAUSE THIS PASS MAKES IT EASY TO MISREAD. The damage chain agrees with the authority at every index and the whole-game clause is RED at 46 games. Both are true and they are not in tension: this table is a per-hit comparison of one calculation, and the whole-game figure is a comparison of boards after a sequence of turns in which address, order, stream and residual all get a vote. A defect of ADDRESS or of STREAM is invisible here by construction, which is what the previous two versions found on the confusion road. A clean 0 of 6,000 is evidence about the formula and about nothing else.

5.251.0 - NO DAMAGE STAGE MOVED, NO STAGE'S MULTIPLIER CHANGED, AND NO DAMAGE VALUE WAS RE-MEASURED. THIS RELEASE IS THE DEFECT REGISTER AND THE COUNTERS. Nothing in this document's stage table, its multiplier classes or its ordering changes. No mechanic was altered in `engine/medicham2-browser.js`. The only edits recorded under this version inside the simulator are counter DECLARATIONS - fields added to the object they were already being incremented on - which change no base power, no multiplier, no roll index and no stage order. The final 85-to-100 per-cent band and its sixteen indices are untouched.

WHY A DECLARATION-ONLY EDIT STILL BELONGS ON THIS PAGE. Four counters were reading `NaN` because they were incremented into an object that never declared them, and one of them went out as `null` in two published artifacts. A counter is how a wire proves it fired; a `NaN` counter is a wire whose proof was fictional. None of the four sits inside the damage chain this table walks, so no figure here is affected - but the general point is the same discipline this document applies to a multiplier, applied to the instrument instead: a value that cannot say where it came from is not evidence, whether it is a stage constant or a fired-counter total.

THE CURRENCY OF THE STAGE-BY-STAGE DIFFERENTIAL IS STILL OWED, ON THE SAME TERMS AS THE PREVIOUS TWO VERSIONS. `tests/test-engine-diff.js` was NOT re-run in this pass. The stage table below is dated evidence from the release it names and is not a current measurement; it is superseded by the next run of that instrument, not by this paragraph. The offered reason at 5.243.0 and 5.244.0 - that nothing feeding it had moved - is not claimed here either, because the simulator's bytes did move even though its behaviour did not, and *the behaviour did not change* is an argument rather than a measurement.

WHICH ARTIFACT HOLDS WHICH WHOLE-GAME FIGURE - UNCHANGED, AND NOT RE-DERIVED BY THIS PASS. `data/verification/fix-batch-M6instr-defog.json` still holds board-material 46 of 961 and protocol 141. `data/game-differential.json` is the artifact the gate clause reads; it was not rewritten and still holds 77 of 961. Name the artifact whenever either is quoted. Neither number moved in this version and neither may be read as evidence that it did.

5.250.0 - NO DAMAGE STAGE MOVED AND NO STAGE'S MULTIPLIER CHANGED. ONE OF THE TWO FIXES IS A DAMAGE VALUE AND IT IS STILL NOT A STAGE: IT IS WHICH RANDOM STREAM THE ROLL COMES OUT OF. Nothing in this document's stage table, its multiplier classes or its ordering changes. The confusion self-hit is not a move and does not walk this chain: the authority computes it inside `getConfusionDamage` and calls the random multiplier DIRECTLY, never entering the wrapped damage function this table is derived against. The fix wraps that method in the instrument and moves the simulator's draw onto the same stream. The final 85-to-100 per-cent band, its sixteen indices and every multiplier above it are untouched, on both roads. The second fix, Defog, is a field sweep and touches no damage stage at all.

THE ONE THING A READER OF THIS TABLE SHOULD TAKE FROM THE PASS. A shared die read in opposite directions is indistinguishable, at the stage table, from two independent dice - and it is worse than independent, because it is anti-correlated. That is a defect of ADDRESS and of STREAM, not of arithmetic, and no stage-by-stage comparison of multipliers can see it. This is the second time in two passes that the confusion road has produced a damage disagreement with every multiplier in this document correct.

WHICH ARTIFACT HOLDS WHICH WHOLE-GAME FIGURE. `data/verification/fix-batch-M6instr-defog.json` holds board-material 46 of 961 and protocol 141. `data/game-differential.json` is the artifact the gate clause reads; it was not rewritten and still holds 77 of 961. Name the artifact whenever either is quoted. The session's full sequence is 77, then 61, then 50, then 53, then 46, and the step that rose is left in.

THE CURRENCY OF THE STAGE-BY-STAGE DIFFERENTIAL IS STILL OWED, AND IT IS STILL WORSE THAN OWED. `tests/test-engine-diff.js` was NOT re-run in this pass, and the simulator moved again, so the reason offered at 5.243.0 and 5.244.0 - nothing that feeds it moved - does not apply here either. The stage table below is dated evidence from the release it names and is not a current measurement. It is superseded by the next run of that instrument, not by this paragraph.

5.249.0 - NO DAMAGE STAGE MOVED AND NO STAGE'S MULTIPLIER CHANGED. THE ONE FIX IS AN ADDRESS AND NOT A MULTIPLIER, AND IT SITS BESIDE THE DAMAGE CHAIN RATHER THAN INSIDE IT. Nothing in this document's stage table, its multiplier classes or its ordering changes. The correction is to WHERE the confusion self-hit takes its two draws from, not to what either draw is worth: `data/conditions.ts` writes `this.activeTarget = pokemon` between the roll for the self-hit and `getConfusionDamage`, so the authority's two draws sit at two different addresses when the confused body has clicked at a foe, and this engine used one address for both. The confusion self-hit does not run through the stage chain this document tabulates - it is `getConfusionDamage`, a fixed 40-power typeless hit - so no row here is affected. **THE ONE THING A READER OF THIS PAGE SHOULD CARRY AWAY IS THAT THE INSTRUMENT HALF IS STILL OPEN AND IT IS A DAMAGE-ROLL ADDRESS QUESTION.** With the addresses matched, the two engines now draw the same value and read it in OPPOSITE directions: the comparator returns the raw index where this engine takes the mirrored one. That cannot be flipped from the engine side without the bottom-tie corner parting on every self-hit, and it is worth all 14 games. Until it is done, the per-hit stage differential and this whole-game figure are answering different questions about the same draw.

THE CURRENCY OF THE STAGE-BY-STAGE DIFFERENTIAL IS STILL OWED, AND IT IS STILL WORSE THAN OWED. `tests/test-engine-diff.js` was NOT re-run in this pass, and the simulator moved again, so the reason offered at 5.243.0 and 5.244.0 - nothing that feeds it moved - does not apply here either. The pinned artifact stranded by the previous release cut has not been regenerated by this pass. Whether `data/engine-diff.json` still describes this engine is unknown rather than merely un-re-checked, and no figure on this page may be quoted as current on the strength of the table alone.

WHICH ARTIFACT HOLDS WHICH WHOLE-GAME FIGURE. `data/verification/fix-batch-M6-sidesel.json` holds board-material 53 of 961 and protocol 154. `data/game-differential.json` is the artifact the gate clause reads; it was not rewritten and still holds 77 of 961. **SUPERSEDED 2026-09-08, narration batch R.** The second sentence was true when it was written and is no longer: `data/game-differential.json` has been rewritten by every whole-game run since, and it no longer holds the protocol figure quoted above at all. The figure is historic and belongs to the verification artifact named beside it; what the gate clause prints today is what `node engine/status.js` prints, and nothing here is restated as current. Name the artifact whenever either is quoted.

5.248.0 - NO DAMAGE STAGE MOVED AND NO STAGE'S MULTIPLIER CHANGED. THE ENGINE BYTES UNDER THIS TABLE MOVED AGAIN, AND NONE OF THE THREE FIXES IS INSIDE THE DAMAGE CHAIN. Nothing in this document's stage table, its multiplier classes or its ordering changes. The three corrections in `engine/medicham2-browser.js` are a refusal evaluated against the wrong aim (Sucker Punch into a redirector), a `stall`

die addressed as two independent coins rather than one shared one, and a contact ability transfer announcing on the wrong body. The first two decide WHETHER a move resolves at all and the third decides WHOSE boost is announced; none of the three changes a damage multiplier, a roll index or a stage order. The board-material whole-game figure fell 61 of 961 to 50 of 961 and protocol first-divergence fell 161 to 150 across that change, on identical pins with the engine release f3504e5f88d6 to 9b449a41c865 .

THE CURRENCY OF THE STAGE-BY-STAGE DIFFERENTIAL IS OWED, NOT ASSERTED, AND IT IS NOW WORSE THAN OWED. tests/test-engine-diff.js was NOT re-run in this pass and the simulator moved again, so the reason offered at 5.243.0 and 5.244.0 — *nothing that feeds it moved* — does not apply. Cutting the new release additionally STRANDED the pinned engine-diff artifact: it names a release the tree has moved past, so its clause cannot speak until it is regenerated. That is the pin guard working rather than a regression, and it means the per-hit result is a figure to WITHHOLD and re-measure. node engine/provenance.js decides that question; this document does not restate the per-hit number.

WHICH ARTIFACT HOLDS WHICH WHOLE-GAME FIGURE. data/verification/fix-batch-M5M7M8.json holds board-material 50 of 961. data/game-differential.json is the artifact the gate clause reads; it was not rewritten and still holds 77 of 961. Name the artifact whenever either is quoted.

5.247.0 - NO DAMAGE STAGE MOVED AND NO STAGE'S MULTIPLIER CHANGED, BUT THE ENGINE BYTES UNDER THIS TABLE DID MOVE, AND ONE OF THE THREE FIXES SITS ADJACENT TO THE DAMAGE PATH RATHER THAN INSIDE IT. Nothing in this document's stage table, its multiplier classes or its ordering changes. What changed is engine/medicham2-browser.js, in three places, taking the board-material whole-game figure 77 of 961 to 61 of 961 on identical pins (release 8ad06030e129 → f3504e5f88d6). **THE ADJACENCY IS WORTH STATING PRECISELY, BECAUSE IT IS EASY TO OVERSTATE IN EITHER DIRECTION.** The multiaccuracy fix changed how many ARRIVALS a volley lands - the authority rolls accuracy per arrival and stops at the first miss, and Triple Axel and Population Bomb are the two carriers in this regulation, derived rather than recalled. It did not change what a single arrival is priced at. Every stage below prices ONE hit, so no stage moved; a volley's total is that price times the number of arrivals, so the whole-game consequence follows from a COUNT and not from a formula. The other two fixes - a non-permanent forme not reverting on switch-out (clearVolatile ends with setSpecies(baseSpecies), sim/pokemon.ts:1564) and a choicelock never cleared - touch no damage stage at all, though a forme carries base stats, so a forme left standing prices later hits off the wrong body. **AND THE MULTI-HIT FAMILY IS THE PER-HIT DIFFERENTIAL'S OWN DECLARED BLIND SPOT,** which is the honest reading of why a volley defect of this size was found by a whole-game comparison rather than by the stage table: the per-hit comparison skips the multi-hit tag by construction and prints that skip on every run.

THE CURRENCY OF THE STAGE-BY-STAGE DIFFERENTIAL IS OWED, NOT ASSERTED. tests/test-engine-diff.js was NOT re-run in this pass, and the reason given at 5.243.0 and 5.244.0 - *nothing that feeds it moved* - is not available this time, because the simulator itself moved. Whether data/engine-diff.json still describes the engine that exists is a question for node engine/provenance.js, which compares content rather than mtimes. This document does not restate a per-hit result measured on engine bytes that have since changed.

WHICH ARTIFACT HOLDS WHICH WHOLE-GAME FIGURE. data/verification/fix-batch-M1M3M4.json holds board-material 61 of 961. data/game-differential.json is the artifact the gate clause reads; it was not rewritten in this pass and still holds 77 of 961. Both are correctly measured on identical pins, and only the second is published.

5.246.0 - NO DAMAGE STAGE MOVED AND NO DAMAGE NUMBER MOVED. THE SECOND-ORDER NOTE PUBLISHED IN 5.245.0 IS RETRACTED, BECAUSE THE INSTRUMENT BEHIND IT WAS WRONG. Nothing in this document's stage table, its multiplier classes or its ordering changes. What changes is a note attached beneath it. The 5.245.0 block below states that *"every body in this engine that loses an item gets Unburden's speed doubling"* and reasons from it about turn order. **That sentence is withdrawn;** the block stands as published and is superseded from here rather than rewritten. engine/medicham2-browser.js:14770 is the ENTRY GUARD only, and the multiplier on the next line, :14772, is gated on TAGS.param('ability', m.ability, 'speedOnItemLoss') — a parameter the ability block of data/tags.json gives to exactly one key, unburden. A census control already on record reads ability none 187,187 for the arm that must not move against Unburden 187,374 for the arm that must. **IT SURVIVED LONG ENOUGH TO BE PUBLISHED BECAUSE OF AN INSTRUMENT DEFECT, NOT AN ENGINE ONE.** tests/probe_leaf_widening.js:277 compared its own stand-in for *this body has lost an item* against the authority's unburden volatile, and never read this engine's Speed; an agreement therefore said that both bodies had lost an item and nothing else. **An instrument can be wrong before the engine is** — the discipline this document applies to a damage stage, applied to the ruler instead of to the game. **WHAT REMAINS IS NARROWER AND STILL TOUCHES NO STAGE HERE.** ROADMAP #535: the doubling is recomputed from the body's CURRENT ability rather than held from the moment the item was lost, so an Unburden acquired afterwards doubles here and not in the authority, with Skill Swap the reachable door. Filed **INSTRUMENT OWED** — nothing decides it yet. Speed still enters no stage of this document's chain; it decides TURN ORDER, so a damage figure that moves because of it has had no damage stage change under it. tests/test-engine-diff.js was not re-run in this pass and no damage figure in this document is re-derived by it.

5.245.0 - NO DAMAGE STAGE MOVED AND NO DAMAGE NUMBER MOVED, AND FOR THE FIRST TIME data/engine-diff.json CAN PROVE WHICH ENGINE PRODUCED THE ZERO. Nothing in this document's stage table, its multiplier classes or its ordering changes. What changed is the artifact's PROVENANCE: engine-diff was regenerated and now carries 26 source_digests plus a showdown_commit, read back field by field rather than trusted, behind one door (engine/pin_guard.js) that withholds a figure when the pin is absent, wrong or unverifiable. **The measured figure is unmoved: 0 of 6,000 at the midpoint and at both corners.** That regeneration was impossible for most of the day: a guard added hours earlier ran at MODULE LOAD off the whole process's argv and killed tests/roster.js --write and engine/all_mechanics_fire.js --write at exit 2 — the SKIP code — so the runner would have reported them politely skipped rather than failed; it now tests require.main === module. **THE WHOLE-GAME EVIDENCE THAT SITS BESIDE THIS TABLE IS NOW A DIFFERENT QUANTITY, AND THAT IS A REVERSAL.** The gating whole-game clause counted protocol first-divergence and printed 167 of 961; it now counts BOARD-MATERIAL games, state.games less state.games_board_never_diverged = 961 - 884 = **77 of 961 (8.0%)**, with narration carried by its own non-gating clause at 167. Any sentence in the dated blocks below that reads 167 as *the* whole-game number is superseded here rather than rewritten, and neither figure may be stated without naming which quantity it is. The gate reads CLOSED - 1 of 8 GATING clauses fail. **ONE SECOND-ORDER NOTE OF THE KIND THAT LOOKS LIKE A DAMAGE CHANGE LATER, AND IT IS NOT ONE.** engine/medicham2-browser.js holds no Unburden state: effSpeed recomputes the doubling from _hadItem && !m.item (:14770), so **every body in this engine that loses an item gets Unburden's speed doubling** — a Knock Off, a consumed berry, a spent Focus Sash. Speed enters no stage of this document's chain. It decides TURN ORDER, so it changes which hit is dealt from what remaining HP and which body is still standing to be hit at all; a damage figure that moves because of it has not had a damage stage change under it. Filed as an ENGINE defect owed a register row, not fixed here. tests/test-engine-diff.js was not re-run beyond the regeneration above, because nothing that feeds the damage chain moved: the only engine-adjacent edit this version is engine/board_state.js, the COMPARATOR, which is not one of the 26 frozen sources and left the release id at 8ad06030e129.

5.244.0 - NO DAMAGE STAGE MOVED AND NO DAMAGE NUMBER MOVED. THE RULER OVER THE DAMAGE TABLE STOPPED LISTING THE FIELDS IT HASHES AND NOW DERIVES THEM, AND THE DERIVATION FOUND A SECOND BLIND CLASS THE LIST COULD NOT HAVE FOUND. Nothing in this document's stage table, its multiplier classes or its ordering changes, and tests/test-engine-diff.js was not re-run because nothing that feeds it moved: no engine byte changed in this pass. The change is again to a RULER rather than to the formula, and it is the SECOND pass over the same ruler, which is the part worth recording here. **WHY 5.242.0 WAS NOT THE FIX, STATED AS A REVERSAL.** That version repaired two named terms in engine/feature_fixture.js's tableDigest() — typing, which was hashed under a field name no row carries, and weight, which was not hashed at all — and this

page recorded it as done. It was INSTANCE-level: the hashed set was still an ENUMERATED LIST, so a field added to the damage table tomorrow goes unhashed exactly as typing did, and the class of failure was re-armed the moment the two instances were closed. The hashed set is now DERIVED from the rows' own keys, minus a declared exclusion map of six entries. **THE SECOND BLIND CLASS, WHICH ONLY THE DERIVATION COULD SURFACE.** An ABSENT field hashes identically to a field that is PRESENT AND EMPTY. So deleting a body's entire moveset moved the digest not at all — live on **4 rows for mv and 10 for item**. Both reach this document's chain: mv decides what a body can throw and therefore which base power, category and type ever enter the formula, and item sits in the multiplier walk. A table change on either could have passed under a fitted vector with the alarm silent. **THE DIGEST VALUE IS UNCHANGED AT 9d289cf77e24**. This adds no third reason to restamp. The two reasons recorded at 5.242.0 — the damage table was regenerated AND the ruler that measures it changed — are still the two, they still cannot be separated by a single restamp afterwards, and the verdict remains RESTAMP-not-refit and the owner's call. **ONE MORE THING THAT COULD NOT SEE THIS TABLE.** tests/test-artifact-keys.js, the check written to catch a hand-typed lookup, sliced to 8 keys, capped depth at 3 where the deepest real object is 10, never descended arrays and exited 0 when the engine data failed to load — so MC.priors, **230 keys**, was never inspected once. No damage number moves because of the repair; what moves is that a wrong lookup in that table is now visible. **A SILENT RULER AND A RULER THAT CANNOT SEE PRINT THE SAME OUTPUT**, which is why both of these were found by mutation rather than by reading, and why the mutation pilot this version records — **83 mutants, 57 killed, 26 survived** — was aimed at the gates and not at the engine: the damage path has the official simulator as an oracle and the rulers over it have none. **5.243.0 - NO DAMAGE STAGE MOVED AND NO DAMAGE NUMBER MOVED, BUT THE WHOLE-GAME EVIDENCE THAT SAT BESIDE THIS TABLE WAS MEASURED WHERE THE GAMES DO NOT END.** Nothing in this document's stage table, its multiplier classes or its ordering changes, and tests/test-engine-diff.js was not re-run because nothing that feeds it moved: the stage-by-stage damage differential is a per-hit comparison and is not steered by a driver at all. What changes is the standing of the OTHER instrument this document has leaned on for whole-game confidence. The whole-game clause of the gate was answered by an artifact from the census-coverage-seeking/v1 driver, which seeks census coverage rather than winning: 17 of 961 games reached a result (1.8%), 944 stopped at the 12-turn cap with both sides standing, and 0 boards parted. On identical pins — release 8ad06030e129, cap 12, pool 0d103fb9fa87, census pin 9446a684709d, 961 games of a 1200 PAIR budget, the driver the only difference — the empirical-click/v1 driver reaches a result in 474 of 961 (49.3%) and 77 boards part. The corrected whole-game figures are therefore 77 of 961 (8.0%) board-material and 168 protocol first-divergence, and the gate reads CLOSED with 1 of 8 clauses failing. Any sentence anywhere that treated a clean whole-game differential as corroboration for this damage path is superseded, including in this document's own dated blocks below, which are kept as written. **WHY THAT MATTERS ON THIS PAGE SPECIFICALLY.** A damage error that only shows itself after a body faints, after a forced switch, or in the last third of a game is invisible in a population where 944 of 961 games stop at the turn cap with both sides standing. The per-band damage agreement recorded in this document is a DIRECT hit-for-hit comparison and stands on its own evidence; the whole-game reading is a different instrument and no longer corroborates it. Nothing was tuned. **5.242.0 - NO STAGE MOVED AND NO DAMAGE NUMBER MOVED; THE ALARM ON ONE DAMAGE INPUT HAD NEVER BEEN ARMED.** Nothing in this document's stage table or multiplier classes changes. The change is to a RULER rather than to the formula: engine/feature_fixture.js's tableDigest() decides whether the damage table underneath a fitted vector has moved, and it was blind to two of that table's fields. It hashed m.ty - present on 0 of 322 rows, because this table spells typing t - so the term was a constant null and a TYPE change never reached it. It did not hash wt at all. **WEIGHT BELONGS IN THIS DOCUMENT BECAUSE WEIGHT IS AN INPUT TO THE DAMAGE NUMBER, NOT A LABEL BESIDE IT.** Six entities in this format read weight or change it - Low Kick, Grass Knot, Heavy Slam, Heat Crash, Heavy Metal and Light Metal - so a species weight edited in the table reaches the damage number at \$1 stage 1, the BasePower chain. Typing is the same exposure ten rows further down the same pipeline: \$1 stage 11, type effectiveness, reads what t says. Both fields are hashed now. **THE BLINDNESS WAS SHOWN BY MUTATION, NOT BY READING.** Mutating one row's t did not move the digest, nor did wt; mv and st did, which is the control that proves the harness can see a change. After the fix all four move. Inventing a ty field moved the digest BEFORE the fix, proving the term was live in the hash and the data had never populated it - an absent field and a present-but-empty field hash identically, so nothing short of a mutation could have told them apart. **NOTHING IN THE DAMAGE PATH MOVED.** The digest moved 1bda9df11d73 -> 9d289cf77e24 **with no change to the table itself**, so no stage, no multiplier and no damage number in this document is affected in either direction, and no damage measurement was re-run or needed re-running. **NO RESTAMP AND NO REFIT WERE RUN, BY THE OWNER'S EXPLICIT DECISION** - MAG stays paused until MEDICHAM is correct, and that is a decision rather than an omission. Full account: docs/_reports/2026-09-03-table-digest-blind-fields.md. **5.241.0 - NO STAGE MOVED, NO DAMAGE NUMBER MOVED ON THE DIRECT ROAD, AND ONE ROAD GAINED A STAGE IT NEVER HAD.** The delayed payout (data/conditions.ts:415 -> trySpreadMoveHit) takes the authority's FULL step list, so the crit step at sim/battle-actions.ts:1156 -> :1636-1642 applies to it exactly as to a direct click, with critMult = [0,24,8,2,1] at :1633, the x1.5 at data/mods/champions/scripts.ts:222 and the |-crit| line at :285. This engine drew no crit for that road at all - not rarely, never - so the stage was ABSENT rather than misplaced. It is now present, and it is present at the CORRECT position: the payout is re-priced through the certain-crit path (\$3, the road that already passes at both endpoints), not multiplied onto a number that has already been rolled, STAB'd, type-charted and chain-spent, which is the battle loop's rolled-crit defect \$3 still records. **The evidence is the unwired-knob signature, and it is the reason a stage can be absent for months without a divergence naming it:** handed a crit-certain die and then a crit-impossible one, the authority answered 72 with the line and 48 without while this engine answered 69 both times. **Nothing in \$1's stage table changes**, and no multiplier moved class in \$2. **The second fix is downstream of the formula, not in it.** A hit that overkills a Substitute doll now books what the DOLL absorbed rather than what the BODY could have taken (data/moves.ts:18341-18357, no Champions override): the damage the formula produced is unchanged, and what changed is the quantity recoil and drain are paid FROM - recoil -25 became -12, drain +62 became +21, both now equal to the authority. **THE CORNER DAMAGE DIFFERENTIAL WAS NOT RE-RUN IN THIS PASS** - the machine was in light mode - so the previous release's figure must NOT be read as covering this change. A road that gained a crit stage can move a damage number, which is exactly what that instrument exists to see; the command is in the report's OWED block and the figure is withheld until it runs. **5.240.0 - NO DAMAGE STAGE MOVED AND NO DAMAGE NUMBER MOVED.** This version changes whether Electric Terrain lets a sleep or a drowse land. Nothing in the damage chain is touched, and tests/test-engine-diff.js was re-run anyway on the same seed as a control: 6,000 compared, 6,000 agreed, 0 disagreed. One second-order note of the kind that looks like a damage change later: a body that stays awake keeps acting, so games under Electric Terrain can now run differently from that point - that is the board changing, not the formula. **5.239.0 - NO DAMAGE STAGE MOVED AND NO DAMAGE NUMBER MOVED.** This version changes where a hazard punish puts its layer and when a weather punish is allowed to set its sky. Neither touches the damage chain, so tests/test-engine-diff.js was not re-run and its 0 of 6,000 at all sixteen corners stands unchanged. One second-order note, recorded because it is the kind of thing that looks like a damage change later: Sand Spit now sets sandstorm over a standing sun or rain, so the sun/rain Fire and Water multipliers stop applying from that point in those games - that is the weather changing, not the formula. **5.237.0 - NO DAMAGE STAGE MOVED AND NO DAMAGE NUMBER MOVED.** This version changes the accuracy chain, which decides WHETHER a move connects and sits above every damage step; tests/test-engine-diff.js was therefore not re-run and its 0 of 6,000 at all sixteen corners stands unchanged. Recorded here because the accuracy stage table (3+n)/3 is a near neighbour of the stat-stage table (2+n)/2 and the two are easy to confuse - they are separate constants and only the accuracy one was touched.

5.236.0 - NO STAGE MOVED AND NO DAMAGE NUMBER MOVED. This version changes how many times the King's Rock flinch die is drawn on a multi-hit move (once per landed arrival, not once per click). The draw is a SECONDARY, taken after the damage is already applied, so no stage in this document is touched and tests/test-engine-diff.js was deliberately not re-run - its standing result is 0/6000 at all sixteen corners, seed 20260804, 2026-08-29, with the unchanged skipped_multihit 134 scope caveat.

5.235.0 - NO STAGE MOVED, NO DAMAGE NUMBER MOVED, AND NOTHING IN THIS DOCUMENT CHANGES. This version adds the collector for the next regulation - a detector that derives the target Showdown format at run time instead of holding a constant, and one store per format id - and repairs the per-game format tag in engine/durable-ingest.js, which returned a literal for every Champions tier. Neither touches the damage path: no multiplier, no stage, no

rounding rule and no order of operations was read or edited, and no figure here was re-measured. It is recorded only so the version line does not assert a damage change it did not make. Full account: `docs/_reports/2026-08-31-next-regulation-ingest.md`.

5.234.0 - NO STAGE MOVED, NO DAMAGE NUMBER MOVED, AND TWO DAMAGE DIVERGENCES BECAME VISIBLE THAT HAD BEEN LABELLED IMPOSSIBLE. Nothing in this document changes. The divergence annotator - the instrument that tags each disagreement with whether the entities it names are legal in this format - was resolving a token to the first dex hit, and `| -damage|p1a:floette|74/149` against `92/149` names Floette-Mega / Floette-Eternal, both legal, through a base spelling that is `Past` and `tier: 'Illegal'`. Both such rows are `board_parted: 1`, `DIFFERENT-END-STATE`, and both had been binned `cannot_occur_in_format: true`. **They are -damage field 3 MAGNITUDE divergences and nothing here yet says which stage they belong to** - no stage was investigated, no multiplier was touched, and no figure in this document was re-measured. They are recorded here so that the next damage-stage pass has them on the list rather than filtered out of it. Full account: `docs/_reports/2026-08-31-annotator-entity-kind.md`.

5.233.0 - NO STAGE MOVED, AND A BASE POWER DID - WHICH IS THE STAGE BEFORE THE STAGES. Low Kick, Grass Knot, Heavy Slam and Heat Crash carry no printed base power and get their real one from a `basePowerCallback` keyed on weight; the Champions mod overrides none of the four, so all four inherit mainline verbatim, and the kg-converted bracket table the engine reads agrees with the hectogram table in the authority. Ten rows of the mon table carried no `wt`, so `effWeight` returned null and all four fell through to the dex base power on a body BUILT at one of them, and to the weight of the body that LEFT the field on a forme change. The artifact is regenerated and the generator's own row census now reports no weightless row. **The multiplier chain is untouched and no damage FORMULA changed;** what changed is the number that enters it as base power. Measured on a real turn against a fixed-base-power move of the same type and category, so every multiplier the forme change moved cancels in the ratio (`data/mechanics-census.json`, `move/variablePower`): Skarmory 50.5->40.4 kg steps the Low Kick / Brick Break ratio DOWNWARD by 25.8%, the only such crossing in this format; Victreebel 15.5->125.5 kg steps 142.4%; and Falinks 62->99 kg crosses no bracket and steps 2.8% identically before and after, which is the cleared control. On the built-at door the three Gourggeist sizes at 9.5, 14 and 39 kg land on 2.00x and 3.00x of the smallest, against a Gourggeist that already carried its weight at 12.5 kg and 1.00x. **The pool's one moved damage value is the RATIO family and not the bracket table** - a Heat Crash at a freshly mega-evolved Falinks, whose user-over-target ratio crosses a bracket even though its target weight does not. The arithmetic and the protocol lines are in `docs/_reports/2026-08-30-engine-data-regen.md`.

5.232.0 - NO STAGE MOVED AND NO DAMAGE NUMBER MOVED; HP MOVED, AND IT IS A HEAL ON THE ATTACKER RATHER THAN A STAGE. Bug Bite and Pluck now make the thief eat the berry they took, so a stolen Citrus pays `maxhp/4` to the ATTACKER through `berryForceEat` - the same shared `singleEvent('Eat')` implementation Stuff Cheeks, Teatime and Cud Chew already use - and the 18 resist berries pay nothing, because their `onEat` body is empty in the authority too. Nothing in `dmgRange` or in the multiplier chain was touched: the resist berry's own halve is applied where it always was, and the probe's Chople arm asserts the thief's HP does NOT move on a berry with an empty effect, which is what separates the heal from a stage. **The one damage-shaped thing still owed here is Ripen's SECOND halve**, which is a `chainModify(0.5)` at `onSourceModifyDamagePriority: -1` gated on `abilityState.berryWeaken`. Measured at 94 against a required ~47 on an Ice Beam into an Appletun holding a Yache, with `MEDFAILS.damageReduceUnknown` naming `ripen/null` - the reader refuses and says so rather than defaulting on. It is blocked on a `tag_dex.js` regeneration, not on the diagnosis.

5.231.0 - NO STAGE MOVED AND NO DAMAGE NUMBER MOVED; HP MOVED, AND THAT IS A HEAL RATHER THAN A STAGE. The type-resist berry's halve is applied by `dmgRange` as a pure read and always was - the probe's empty-hand arm asserts it exactly, taking 296 against the berry arm's 148 both before and after this change - so what moved at that site is only WHO gets told the berry was eaten. `consumeBerry` now runs there, which raises `runEvent('EatItem')`, and a Cheek Pouch holder therefore takes a `maxhp/3` heal between the `-enditem [eat]` and the `-enditem [weaken]`, exactly where `chopleberry.onSourceModifyDamage`'s `if (target.eatItem()) { ... }` puts it. That heal lands during the damage CALCULATION and before `spreadDamage` moves any HP, which is the authority's own position and not a choice. **One thing worth stating for the next reader of this file:** Ripen's `onSourceModifyDamage` is a SECOND `chainModify(0.5)` gated on `abilityState.berryWeaken`, written from `onEatItem`. This engine's `damageReduce` row for Ripen carries `onlyWhen: null` and the reader REFUSES it rather than defaulting on, counting `MEDFAILS.damageReduceUnknown` - so that multiplier is still absent from this stage, and it is absent DELIBERATELY. Raising the event is its prerequisite; wiring the multiplier is filed as its own batch. `tests/test-engine-diff.js` was NOT re-run: it calls `moveHit` once, has no `Update` pass and no residual berry, so it is structurally blind to every road this batch touched - an argument, not a measurement, recorded as one.

5.230.0 - NO STAGE MOVED, AND ONE DAMAGE INPUT DID — DECLARED RATHER THAN LEFT TO BE FOUND. Both changes decide WHERE a line is written: the busted-disguise `detailschange` and its `maxhp/8` chip move from the hit to the `Update` at the foot of the hit, and Cheek Pouch's heal moves below the berry's own. Neither touches the packet vector, the roll index or the crit decisions, all of which are drawn in `_stepDamage` above all of this. **The declared change: on a single-arrival click into an intact Disguise, dmg now reads 0 rather than the chip.** That is `onDamage`'s own return (`data/abilities.ts:962-967` returns 0, so `damage[i]` is 0) and the chip is a separate `this.damage(baseMaxhp/8, pokemon, pokemon, species)` whose source effect is a SPECIES and not a Move — so no Focus Sash, no Endure and no recoil may answer it, and those three blocks sit below the absorb and read `dmg`. It is strictly closer to the authority than the value that used to sit there, it is reachable only on a body an intact Disguise is absorbing for, and it is stated here because it is a state change riding in an ordering batch. **The HP is unchanged and the probe asserts it:** the holder ends on `maxhp - maxhp/8` on the spread arm and on the plain arm, and the Cheek Pouch body ends on the same total with the fix and without it — only what the berry's own `-heal` line REPORTS moved (370/596 now against 568/596 before, the post-both total), which is asserted against a no-ability control. **A pre-existing damage divergence was measured in the same pass and filed rather than fixed:** a type-resist berry never routes through `consumeBerry`, so Cheek Pouch pays nothing for it — Close Combat into a Cheek Pouch Maushold holding a Chople Berry emits `-enditem [eat]`, `-enditem [weaken]`, `-damage` and no `-heal`. `tests/test-engine-diff.js` was NOT re-run: it calls `moveHit` once, has no `Update` pass and no residual berry, so it is structurally blind to both changes — which is an argument and not a measurement, and it is recorded as one.

5.229.0 - NO STAGE MOVED AND NO DAMAGE NUMBER MOVED. The resist berry's halve is applied by `dmgRange` as a pure read and always was; only the CONSUMPTION and its two `-enditem` lines moved, from the apply step up into the calculation step where `getDamage` raises `ModifyDamage`. The arithmetic control is in the probe rather than in this sentence: the same board with an empty hand takes exactly double (79 against 158). The `DamagingHit` change moves WHEN a reaction is paid within a volley, never how much: the toll, the boost and the arrival amounts are unchanged, the packet vector and the roll index are drawn in `_stepDamage` before any of this runs, and the per-arrival crit decisions are untouched. **One reachable path is named rather than left to be found:** an attacker killed by an interior arrival's toll now dies mid-volley, and `hitStepMoveHitLoop` breaks there (`scripts.ts:534-537`) while this packet loop tests only the TARGET's HP - so the volley still runs to its drawn length. Nothing in the 961 pinned games stages it. **A pre-existing damage divergence was measured in the same pass and filed rather than fixed:** a multi-arrival volley halves EVERY arrival against a resist berry where the authority halves only the one that ate it (Triple Axel into a Yache Berry reads 1464 -> 1434 -> 1374 -> 1284 with the berry against 1404 -> 1284 -> 1104 without). `tests/test-engine-diff.js` was NOT re-run: it calls `moveHit` once and cannot see a multi-hit at all, which is an argument and not a measurement, and it is recorded as one.

5.228.0 - NO STAGE MOVED AND NO DAMAGE NUMBER MOVED. Both changes decide WHEN a residual chip or a residual counter runs relative to the others, never HOW MUCH it takes: the burn is still `max(1, floor(maxhp/16))`, ordinary poison still a flat eighth, Toxic still the escalating sixteenth, and the Perish Song step ticks a counter and calls `faint()` without computing damage at all. Every one of them sits in the end-of-turn walk, below `_stepApply` and below the whole hit loop, so the packet vector, the roll index and the crit decisions are untouched. The one path by which an ordering change could reach a damage number is

a body dying earlier or later and therefore taking a different number of chips - and the amounts and the totals are unchanged in every staged arm, including a four-body mutual perish wipe that is byte-identical before and after. `tests/test-engine-diff.js` was NOT re-run: it calls `moveHit` once and has no residual walk, so it is structurally blind to this change - which is an argument and not a measurement, and it is recorded as one.

5.227.0 - NO STAGE MOVED AND NO DAMAGE NUMBER MOVED. The change decides WHEN a `boostsOnKO` payment is made, HOW LARGE one payment is, and whether it is made at all once the drain has emptied a side. All three are downstream of every damage step: the step is inserted below `_stepDrainFaints`, which is itself below `_stepApply`, so the packet vector, the roll index and the crit decisions are untouched. The BOOST it grants can reach a later turn's damage, and that is exactly the case the fix removes - a payment made after the battle ended has no later turn to reach. The `single` arm of `tests/probe_afterfaint_boundary.js` is the control that holds this: a single KO on a continuing battle reads `faint,ABIL,BOOST:atk+1` on both engines, before and after. `tests/test-engine-diff.js` was NOT re-run; it calls `moveHit` once and has no faint drain, so it is structurally blind to this change - which is an argument and not a measurement.

5.226.0 - NO STAGE MOVED AND NO DAMAGE NUMBER MOVED. The change decides how many times an `onDamagingHit` REACTOR is raised for a volley that stopped at a KO; it is downstream of every damage step, and the packet vector, the roll index and the crit decisions are untouched. The damage a reactor DEALS is unchanged - `punishesAttacker`'s `fraction` and `dealsDamageTaken` arithmetic is not edited - only the number of times it is charged. The survivor arm of `tests/probe_volley_reactor_count.js` is the control that holds this: identical HP and an identical `-hitcount` on both engines, before and after. `tests/test-engine-diff.js` was NOT re-run; it calls `moveHit` once and skips every `multiHit` move by construction, so it is structurally blind to this change - which is an argument and not a measurement.

5.225.0 - NO STAGE MOVED AND NO DAMAGE NUMBER MOVED. The change decides WHICH SIDE a body that the aim already resolved is looked up on, which is above every damage step; no multiplier is added or removed and no feature read changes. The damaging forced-switch door runs AFTER the hit, so the drag it now performs for an ally-aimed Dragon Tail happens on a board the damage pipeline has already finished with. The two FOE-axis control arms of `tests/probe_ally_forced_switch.js` are line-identical clean and under the knob, which is the same claim measured. `tests/test-engine-diff.js` was NOT re-run - it has no `--out` and would republish the artifact the `0 of 6,000` is read from.

5.224.0 - NO STAGE MOVED AND NO DAMAGE NUMBER MOVED. The change decides WHICH BODY a repeated move is aimed at, not what it does when it lands. No multiplier is added or removed and no feature read changes; the same damage pipeline runs against a different defender. The one adjacent effect is that a spread or `randomNormal` repeat is now PRICED against the recorded slot rather than against the hardest-hit foe — a valuation field on the action, not a stage — and the class gate keeps the loc out of the spread road entirely. The stage-by-stage comparison against the authority is unaffected; `tests/test-engine-diff.js` was re-run and reads `disagreed 0`.

5.223.0 - NO STAGE MOVED AND NO DAMAGE NUMBER MOVED. The change is a REFUSAL: a move that would have queued a second action does not queue it. Nothing in the damage pipeline is touched, no multiplier is added or removed, and no feature read changes. What changes is HOW MANY damage calculations a turn contains — a shielded body no longer takes a swing it was never given. The stage-by-stage comparison against the authority is unaffected and was not re-run.

5.222.0 - NO STAGE MOVED, NO DAMAGE NUMBER MOVED, AND NO FEATURE READ MOVED. This version changes test instruments, one run wrapper and the defect register. It touches no damage packet and no input to any stage in the table below. `data/engine-diff.json` was not rewritten and its modification time is unchanged, so the published **0 of 6,000 at all sixteen roll positions** still stands from 02:49.

CORRECTION TO THE 5.221.0 NOTE BELOW, WHICH IS LEFT STANDING RATHER THAN EDITED. That note gives as its reason for not re-running the differential that the check "has no `--out` and would republish that artifact". That was true when it was written. `--out` **now exists**: it names a path for the run's artifact, refuses any path outside `data/verification/`, and leaves the published artifact alone. So the reason has changed even though the decision has not - the differential still was not re-run for a figure in this pass, and the published residual still stands from 02:49 rather than from today.

For the record, and because it bears on how anyone reads this instrument: the damage residual has never been this check's exit code. `disagreed` is published to the artifact and read by `engine/quarantine.js`. The only three conditions that set a failing status are the accuracy, accuracy-modifier and substitute-bypass conformance sections.

5.221.0 - NO STAGE MOVED, NO DAMAGE NUMBER MOVED, AND NO FEATURE READ MOVED EITHER. The change in this version decides WHETHER THE USER OF A STATUS MOVE LEAVES THE FIELD after that move resolved. It touches no damage packet: `partingshot` has `basePower: 0` and `category: 'Status'`, so no stage in the table below is reached at all on the turn in question, and none of the inputs to any stage changed. Unlike 5.220.0 this batch touches nothing `board.js` reads, so `benchRisk` is unmoved and no refit is owed.

`data/engine-diff.json` was not rewritten and `tests/test-engine-diff.js` was not re-run - it has no `--out` and would republish that artifact. The published **0 of 6,000 at all sixteen roll positions** therefore still stands from 02:49, and it is unchanged by construction rather than by assertion: that instrument compares `dmgRange` outputs and never reaches a `selfSwitch`.

5.220.0 - NO STAGE MOVED AND NO DAMAGE NUMBER MOVED. The change in this version decides WHETHER A MOVE IS REFUSED AT ALL, above every damage step: five priority gates now compare the ability-modified priority instead of the printed constant. A refused move produces no damage packet, so no stage in the table below is reached differently and none of its inputs changed. `data/engine-diff.json` was not rewritten - the published **0 of 6,000 at all sixteen roll positions** still stands from 02:49.

ONE FEATURE READ DID MOVE AND IT IS DECLARED HERE TOO. `clickFragility`'s "blocks priority outright" clause now reads the same number, so `benchRisk` moves for the format's single Gale Wings carrier. That is a FEATURE, not a damage stage - `dmgRange` is untouched - and it is owed a refit at the next release cut.

`tests/test-engine-diff.js` **WAS DELIBERATELY NOT RE-RUN THIS PASS**, because it has no `--out` and would republish that artifact. That instrument compares `dmgRange` outputs directly and never calls a priority gate or `clickFragility`, so it is unchanged by construction; the probe's `nogalewings- bravebird`, `galewings-damaged` and `quickguard-priority0` arms carry HP fractions that are identical between the two engines and on both loads.

5.219.0 - NO STAGE MOVED AND NO DAMAGE NUMBER MOVED. The change in this version decides WHICH BODY a substituted move addresses, above every damage step - a move an Encore forced or a Copycat produced now takes its target from its own target class instead of always from the foes. Nothing above or below the stage table is touched, and `data/engine-diff.json` was not rewritten: the published **0 of 6,000 at all sixteen roll positions** still stands from 02:49.

`tests/test-engine-diff.js` **WAS DELIBERATELY NOT RE-RUN THIS PASS**, because it has no `--out` and would republish that artifact. The probe carries the evidence instead: its `encore-aurasphere` and `copycat-aurasphere` arms are line-identical between the two engines and fail if the far-side draw consumes a different die, which is the only way this change could reach a damage number.

5.218.0 - NO STAGE MOVED AND NO DAMAGE NUMBER MOVED. The change in this version is a SHIELD-GATE one: a shield whose move was substituted mid-turn now passes the queue scan and the consecutive-use roll, where before it passed neither. Nothing above or below the stage table is touched, and `data/engine-diff.json` was not rewritten — the published **0 of 6,000 at all sixteen roll positions** still stands from 02:49.

RE-RUN THIS PASS, and it went to `data/verification/engine-diff.n150.json : tests/test-engine-diff.js reports 0 disagreements over 150 matchups` at seed 20260804. The rc=3 a reader may see is that file's pool advisory rather than a disagreement, A/B verified as pre-existing on release `cc7dca43e395`.

5.217.0 - NO STAGE MOVED AND NO DAMAGE NUMBER MOVED. The change in this version is a TURN-ORDER one: a mid-turn Encore now relocates its target's queued action into the ENCORED move's priority bracket, which is where Champions' own `encore.condition.onStart` puts it. Nothing above or below the stage table is touched - the same move deals the same amount, at a different point in the turn. **RE-RUN THIS PASS, unlike 5.216.0:** `tests/test-engine-diff.js reports 0 disagreements over 150 matchups`, seed 20260804, and it wrote to `data/verification/engine-diff.n150.json - data/engine-diff.json` was NOT touched and the published **0 of 6,000 at all sixteen roll positions** still stands from 02:49. The same run was repeated against `git show HEAD:engine/medicham2-browser.js` swapped into the tree and produced the identical verdict and the identical exit code, so the rc=3 this file's reader may see is the pool advisory for 9 undrawable species and not a disagreement.

5.216.0 - NO STAGE MOVED AND NO DAMAGE NUMBER MOVED. The change in this version is a STATUS REFUSAL: Safeguard now refuses what the target's own partner wrote, which is a `setStatus` and an `addVolatile` question and never reaches the damage chain. `tests/probe_ally_safeguard.js` asserts it explicitly - an ally's Earthquake into the shielded body deals a bit-identical amount on both arms of the Safeguard knob, which is a probe reading and is not a figure from this table. The 6,000-comparison damage differential is expected unchanged at **0 disagreements at all sixteen roll positions**, seed 20260804; it was NOT re-run this pass, deliberately, because `tests/test-engine-diff.js` has no `--out` and would republish `data/engine-diff.json`.

5.215.0 — NO STAGE MOVED AND NO DAMAGE NUMBER MOVED. The change in this version is to a MEASUREMENT INSTRUMENT — the roster's move arm now asks whether a one-turn state ever refused anything — and touches nothing above or below the stage table. The 6,000-comparison damage differential is unchanged at **0 disagreements at all sixteen roll positions**, seed 20260804.

5.214.0 — NO STAGE MOVED, AND NO DAMAGE NUMBER MOVED. The change in this version is a TARGET SELECTION defect, not a damage one: a pivot status move was not offered to the redirection event, so its stat drops landed on the body it named instead of on the redirector. Nothing above or below the stage table was touched. The 6,000-comparison damage differential is unchanged at **0 disagreements at all sixteen roll positions**, seed 20260804, which is the reading this document exists to hold.

5.213.0 — THE GRASS KNOT AND HEAVY SLAM ROWS THE STANZA BELOW CALLED "THE SHARPEST OPEN DAMAGE QUESTION IN THIS DOCUMENT" ARE CLOSED, AND NO STAGE MOVED. Neither was a stage and neither was a roll: base power is computed ABOVE the stage table, and this engine held the body's weight as a build-time constant while the authority rewrites it at every forme change (`Pokemon#setSpecies`, `sim/pokemon.ts:1402`). A mega evolution was therefore priced off the body that left the field — Grass Knot into a Staraptor-Mega read the base forme's 24.9 kg (bracket `>=10`, BP 40) where the authority read 50 kg (bracket `>=50`, BP 80), and Heavy Slam by a Steelix-Mega read a 400/120 ratio (BP 80) where the authority read 740/120 (BP 120). Predicted damage ratios 0.500 and 0.667 against observed 0.478 and 0.677. **The remaining two weight cards ARE roll residue** (Low Kick 0.947, Heat Crash 1.126) and are correctly not a stage question either. The 6,000-comparison damage differential is unmoved at 0 disagreements at both endpoints, seed 20260804. Full account: `docs/_reports/2026-08-29-weight-base-power.md`.

5.212.0 — NO DAMAGE STAGE MOVED, AND THE POPULATION THE -damage field 3 CARDS WERE COUNTED IN DID. The whole-game differential's forced-switch mirror was stopping 42 of 961 empirical-arm games on a harness fault rather than on the game; it now stops 27, and the count of games whose board diverged fell from 135 to 117. Any damage card counted against the 135 is an upper bound until it is re-read against the 117 — including the twelve -damage field 3 cards of `docs/_reports/2026-08-29-empirical-divergence-cards.md`, whose Grass Knot and Heavy Slam rows remain the sharpest open damage question in this document. Nothing in the stage table below was measured on that arm, so no stage verdict here changes.

5.211.0 — THE SPREAD EVERY DAMAGE NUMBER HERE RUNS ON IS ASSIGNED, NOT OBSERVED. A Showdown open team sheet reveals species, item, ability, moves, nature, gender and level and NOT the spread, so `engine/game_differential.js spreadFor(index)` assigns one from the body's slot index: 66 points, a 32 cap, a descending Speed ladder by slot, the remainder to the higher attacking stat and then spilling to Sp. Def then Def, and **nothing into HP** — deliberate, because Champions' Showdown line adds the investment plus 75 for HP while `medicham2's` L50 line has no HP term, so HP points would diverge silently on every body. Both engines are handed the same spread, so a stage disagreement is still a real disagreement; what a clean damage verdict is NOT is a statement about what the ladder actually rolls. `engine/coverage.js` prints this beside the verdict and reads the budget, the cap and the ladder off the driver's source rather than repeating them.

5.210.0 — THE PORY TWO-FEATURE PAIR IS WITHDRAWN: ITS GENERATOR WRITES NO ARTIFACT. `engine/pory_baseline.py` prints a five-arm table and saves nothing, so the material-baseline pair it published on 2026-07-25 never had a source to check it against, and it was scored before that script had a clean-data filter at all. On the clean corpus the comparison is a TIE rather than a loss, measured PAIRED and clustered by game in `data/pory-eval.json`. The withdrawn pair stays in `docs/REVIEW-2026-07-25.md`, the review that measured it. This document does not quote the pair and is unchanged apart from this note.

5.209.0 — STAGE 3 GAINED A MEMBER: FLASH FIRE'S ABSORBED VOLATILE. The blocks below are dated history and are not rewritten. The placement is read off the authority rather than argued: `flashfire.condition.onModifyAtk / onModifySpA return this.chainModify(1.5)` when the move is Fire and the attacker still has the ability, and `data/mods/champions/abilities.ts` has no `flashfire` key, so mainline's handler is the format's unchanged. It therefore folds into the `_aCh` relay beside Guts, Huge Power, Solar Power, Orichalcum Pulse and Hadron Engine and is spent once — NOT into the final `ModifyDamage` chain, which is the mistake \$2 already measured for Thick Fat and for Water Bubble — the disagreement rates are in that section and are not restated here. The multiplier, the boosted type and the stats are read from `typeImmunity.gain.volatileBoost` in `data/tags.json`; no number is typed.

Version: 5.208.0 — 2026-08-28.

5.208.0 — THIS CHAIN GAINED A MEMBER. STAGE 13 NOW CARRIES THE METRONOME ITEM, AND THE STAGE IS DERIVED FROM THE AUTHORITY RATHER THAN CHOSEN. The two blocks below both begin *"nothing in this chain is touched"*; that is no longer true and the change is recorded here rather than by editing them.

WIRE 158 gave the tag `damageMultOnRepeat` its first consumer. The placement is read off the authority, not argued: `data/items.ts:4022` is `onModifyDamage(damage, source, target, move)` returning `this.chainModify([dmgMod[numConsecutive], 4096])`, and `data/mods/champions/items.ts` carries **no** `metronome` key at all — checked against the mod file, not recalled — so mainline's handler is this format's handler unchanged. `data/mods/champions/scripts.ts:293` spends that event as `runEvent('ModifyDamage', pokemon, target, move, baseDamage)` with `pokemon` the ATTACKER, which is why the member reads the ATTACKER's item and sits beside Life Orb rather than beside the resist berries — those are `onSourceModifyDamage` on the defender. Row 13 of the stage table is updated in place.

THE ORDER INSIDE THE CHAIN CANNOT MATTER FOR THIS MEMBER, AND THAT IS A PROPERTY RATHER THAN A HOPE. The ladder is stored in 4096ths, so every step divided by 4096 is a dyadic rational, `_sdTrunc(m*4096)` returns the step unchanged and `ch4096` introduces no rounding of its own — the same property this file already records for x1.5 and x2. Nothing in the engine types a step: the array and its denominator are read from the tag, an unreadable

ladder applies nothing and is counted, and the per-body consecutive-use counter is written at the authority's own `onTryMovePriority: -2` position, below the PP deduction and above the shield gate.

THE DAMAGE DIFFERENTIAL WAS NOT RE-RUN AND THAT IS A GAP, NOT A CLEARANCE. It still reads 0 disagreements of 6000 at each of the sixteen band indices, measured before this member existed. Its declared scope is damage only, and it has never applied a multi-hit move. The evidence for the new member is two dedicated probes instead: a ladder probe pricing rungs 0 to 6, with rung 0 byte-identical to a body holding no item, and a six-turn game probe comparing 394 leaves at each of 7 boundaries. Both were shown red first under `MEDI_NO_METRONOME_LADDER=1`.

5.207.0 — RE-READ AFTER THE CLOSET PASS: NOTHING IN THIS CHAIN IS TOUCHED, AND THAT IS ASSERTED RATHER THAN ASSUMED. The 5.207.0 release closed the last open gate clause by DECLARING one divergence — a Perish Song faint announced above `[upkeep]` instead of below it — through a `kind: 'CLOSETED'` row in `engine/quarantine.js`. **No engine byte moved and no artifact was regenerated.** The damage differential was not re-run because nothing that feeds it changed; it still reads 0 disagreements of 6000 at each of the sixteen band indices, with the same standing scope limit: damage only, and it has never applied a multi-hit move. The stage table below stands unedited.

5.206.0 — RE-READ AGAIN AFTER THE CRLF PASS: NOTHING IN THIS CHAIN IS TOUCHED, AND THAT IS ASSERTED RATHER THAN ASSUMED. The 5.206.0 release restored five withheld gate clauses that had gone blank on a line ending, pinned seventeen frozen sources to LF in `.gitattributes`, and re-ran the roster, the whole-game differential and the staged-mechanics comparison on release `5f3f7141227c`. **No damage stage, no ordering, no rounding rule and no override was changed by any of it.** The damage differential was not re-run because it was never stranded — its gate clause passed throughout — and it still reads 0 disagreements of 6000 at each of the sixteen band indices, with the same standing scope limit: damage only, and it has never applied a multi-hit move. The stage table below stands unedited.

5.205.0 — THE SPRINT IS PAUSED AND THIS FILE IS RE-READ, NOT REWRITTEN: NO STAGE MOVED. The living-docs deferral that ran from 2026-08-10 is over. Across the whole sprint the stage ORDER inside one hit is unchanged, and `tests/test-damage-stages.js` re-reads **1696/1696 exact, 0 at the wrong stage**, across all sixteen rolls and both crit states, with 5 re-derived `CH_EXACT` overrides and 0 wrong. The population moved (this file previously recorded 1728 cells); the verdict did not.

WHAT CHANGED IS OUTSIDE THIS CHAIN, AND BOTH ITEMS BELONG TO THE BATTLE LOOP. The crit is now drawn **once per hit** rather than once per click, which is what the authority does — its loop is the Champions mod's and its die is mainline's, inside the per-hit `getDamage` call, and neither `getSpreadDamage` nor `getDamage` is overridden by the mod. Arrival 2 of a volley no longer inherits arrival 1's crit. Separately, the event die itself gained a finalising mix; before that, consecutive arrivals shared a 16-bucket damage index 89.5% of the time against a correct 6.25%. **Neither touches the stage order this document describes**, and both are why a damage figure measured before 2026-08-27 is void rather than stale.

THE DIFFERENTIAL THAT CHECKS THIS CHAIN HAS NEVER APPLIED A MULTI-HIT MOVE, AND THAT MUST BE SAID WHEREVER ITS FIGURE APPEARS. Read from `data/engine-diff.json`: 6000 compared, 6000 agreed, 0 disagreed, and 0 at each of the sixteen band indices separately. Its own scope field limits it to damage only — no items, no abilities, no turn order, no status duration, no switching — and it records `skipped_multihit` 134 and `skipped_ability_multihit` 17, because the harness calls the authority's single-hit entry point rather than the volley loop. The multi-hit defects corrected during this sprint were invisible to it by construction. **The interior of a multi-hit range remains a single draw across the summed endpoints** rather than N independent ones; that is unchanged by this pass and is still the battle loop's question rather than this chain's.

3.98.0 — NO STAGE MOVED AND NOTHING IN THIS FILE CHANGED. ROADMAP #126 wired Quick Guard onto the priority-refusal gate. That gate sits in the TURN LOOP, above the action-kind dispatch, and never reaches `dmgRange`: a refused move deals no damage at all rather than damage at a different stage. `tests/test-damage-stages.js` is unchanged. The version moves because the CHANGELOG top moved, and this line says why the content did not.

3.97.0 — THE DAMAGE IS A LOOP OVER HITS NOW, AND NO STAGE MOVED. `dmgRange` is a wrapper over `dmgRangeOneHit`; the stage ORDER inside one hit is byte-for-byte what this document already describes, and `tests/test-damage-stages.js` re-reads **1728/1728 exact, 0 at the wrong stage**. What changed is how many times that chain is spent: once per HIT for a move whose base power is a function of the hit index (Triple Axel `20 * move.hit`, Beat Up one packet per eligible ally), and once in total — with the old `_hits` scalar — for everything else, including the rest of the multi-hit family. **The pinned endpoints are the authority's:** `min` is every hit at the 85% randomizer and `max` every hit at 100%, which is what a pin produces in Showdown. **The interior of a multi-hit range is still a single draw** across the summed endpoints rather than N independent ones — unchanged by this pass, and it is the battle loop's question rather than this chain's. Fickle Beam's conditional power left this file's arithmetic entirely: it is a DRAW taken in the battle loop, not a $\times 1.3$ on the base power.

3.96.0 — THREE DEAD LINES LEFT THE ATTACK AND DEFENCE CHAINS, AND ONE LIVE ONE JOINED. The hardcoded Choice Band / Choice Specs / Assault Vest multipliers were permanently false — all three are banned in this format — and are replaced by a derived `statMult` consumer whose only live member here is Light Ball. The chains themselves are unchanged in ORDER and `tests/test-damage-stages.js` re-reads **1728/1728 exact, 0 at the wrong stage**. **The boost-stage difference filed at 3.89.0 is still filed and still not fixed.**

3.95.0 — A DAMAGE ANSWER CHANGED, AND IT IS NOT A STAGE. `dmgRange` now returns **0** against an intact Disguise, because the authority's `onDamage` blocks the move outright and the `maxhp/8` that busts it belongs to the ABILITY. Nothing in the table below moved — no multiplier changed position, and `tests/test-damage-stages.js` re-reads **1728/1728 exact, 0 at the wrong stage** — but this is the first entry here that alters what the function RETURNS rather than where a multiplier applies, so it is recorded as such. **The boost-stage difference filed at 3.89.0 is still filed and still not fixed.**

3.94.0 — NOTHING IN THE TABLE BELOW MOVED. ROADMAP #110's `selfBoost` fix adds the USER's own stat change to two move rows; it is a boost applied after the move, not a multiplier inside it. `tests/test-damage-stages.js` re-reads **1728/1728 exact, 0 at the wrong stage**. **The boost-stage difference filed at 3.89.0 is still filed and still not fixed.**

3.93.0 — NOTHING IN THE TABLE BELOW MOVED. ROADMAP #110's partial-trap fix is a duration COUNTER, not a multiplier, and the chip fraction it carries (1/8) was correct throughout — what changed is that it is now derived from `onStart`'s `boundDivisor` rather than typed. **The boost-stage difference filed at 3.89.0 is still filed and still not fixed.**

3.92.0 — NOTHING IN THE TABLE BELOW MOVED. `tests/test-damage-stages.js` stopped padding its inert slots with `Tackle`, which is `isNonstandard: 'Past'` and not in this format. Those slots never act, so the reading is unchanged: **1728/1728 exact, 0 at the wrong stage**. **The boost-stage difference filed at 3.89.0 is still filed and still not fixed.**

3.91.0 — NOTHING IN THE TABLE BELOW MOVED, AND NO STAGE QUESTION WAS RAISED. ROADMAP #116 is a legality guard on the probe harness, not a multiplier. It changes which bodies may be staged, never where a multiplier applies. Worth one line here for a reason that touches this document directly: the stage table is measured by probes, so a probe staging an entity the format does not contain would put a row in it about a mechanic no game can reach. **The boost-stage difference filed at 3.89.0 is still filed and still not fixed.**

3.90.0 — NOTHING IN THE TABLE BELOW MOVED, AND THE ONE STAGE QUESTION THIS PASS RAISED WAS ANSWERED NO. ROADMAP #103 is a hit COUNT, not a stage. The plausible alternative WAS a stage question — whether the authority's per-hit floor (`n` independent `floor`s) differs from this engine's single `floor(v * n)` — and it was ruled out with arithmetic rather than a preference: `roll()` already returns an integer, so for an integer

count the two expressions are equal for every value. The multiplication at `dmgRange`'s tail is unchanged. **The boost-stage difference filed at 3.89.0 is still filed and still not fixed.**

3.89.0 — NOTHING IN THE TABLE BELOW MOVED, AND ONE NEW DIFFERENCE IS FILED INTO IT. ROADMAP #101 and #102 are a post-damage REACTION and a HEAL; neither is a stage. But wiring Strength Sap's heal off `getStat('atk', false, true)` surfaced a boost-stage difference that IS a damage question: `dmgRange` applies a stage as `Math.floor(x * boostMul(s))`, where `sim/pokemon.ts` MULTIPLIES on a positive stage and DIVIDES on a negative one. The two disagree wherever the float lands just under an integer — at `s = -1`, `x = 3` they give 1 and 2. The new heal uses a helper (`statWithBoost`) that mirrors the authority exactly; `dmgRange` was deliberately left alone in the same pass, because changing it is a damage change and would have needed its own measurement. **Filed here, not fixed.**

3.88.0 — TWELVE MOVES WERE PRICED OFF GENERIC GEN-9 DATA INSTEAD OF THIS FORMAT'S, AND THE BUILDER THAT FIXED THEM WAS ONE RUN AWAY FROM DELETING TEN SPECIES. Trop Kick read 70 where the format says 85, Mountain Gale 100 against 120 — ours low in all twelve, and MAG's own table had the right numbers the whole time, so the two engines disagreed on every one. Asking what a regeneration WOULD do, before running one, turned up 788 destructive changes waiting in the same builder and a header stamp whose regex had never once matched. `buffsHolderOnHit` also gained its condition by derivation — Anger Point only on a critical hit, Justified only on Dark — but **the engine does not read it yet and nothing behaves differently**, which is said here rather than left to look like a fix.

3.87.0 — THE STAGE WAS RIGHT AND THE INPUT TO IT WAS NOT: TWO READERS OF THE WEATHER. Nothing in the table below moved. The BasePower-chain member at `:1991 (weatherScaled)` sits at the correct stage and reads the correct sky, because `dmgRange` shadows the field through `effWeatherOf` at its first statement — which applies the PRIVATE weather a Mega Sol body carries. The battle loop's own type authority, `effMoveType`, did not: it read `field.weather` raw. So the same click was a Fire move to this table and a Normal one to the stage-5 immunity gate, and a Meganium-Mega's Weather Ball priced here at 128-151 dealt 0 to a Ghost. `effMoveType` now CALLS `effWeatherOf`. This is §2's lesson in a new place: the stage-by-stage audit can be clean while a DIFFERENT reader of the same fact makes the whole row unreachable.

3.86.0 — EVERY PUBLISHED ARTIFACT HAS A WRITER NOW, INCLUDING THE ARM MILTANK ACTUALLY RUNS. The tool that answers "is this number still true" could only find a writer by an artifact's literal name beside a write call in two directories — so the mechanics census, the game differential, the interaction matrix and the deliberate roster, which are the four clauses of the MEDICHAM gate, had no row at all. Not ok, not unsafe: absent. The graph goes 115 to 160 artifacts and the unknown set 61 to 16, membership derived through four ranked arms that each record how they matched. UNSAFE rises 13 to 20 because seven artifacts that were always unsafe are now visible; none left the set.

3.85.0 — THE WHOLE SITE WITHHOLDS NOW, AND THE DEPLOYED COPY WAS MISSING THE FILE THAT MAKES IT WORK. Five pages LOADED a quarantined artifact as data instead of quoting its verdict, so the citation checker could not see them at all; seven of the Stadium's fifteen cabinets now go dark, each keeping its seat and its button and answering with the quarantine instead of a number. The file that drives all of it, `quarantine-data.js`, did not exist under `app/` — which is the copy a visitor loads — so every guard there took the healthy path. That is the same failure as the day before, one directory over.

3.83.0 — THE PINCH FAMILY FIRES, AND THE REFUSAL THAT HID IT WAS CORRECT THE WHOLE TIME. 3.80.0 (below) found that Blaze, Torrent, Overgrow and Swarm carry their below-1/3-HP condition as PROSE and that the `damageBoost` consumer refuses any condition it cannot evaluate. That refusal is #92's own rule and it is not removed. What changed is upstream: `engine/tag_dex.js` now derives the gate by SHAPE out of Showdown's `attacker.hp <= attacker.maxhp / 3` into `{cond:'hpFraction', of:'self', cmp:'<=', num:1, den:3}`, and `condHolds` evaluates it in INTEGER arithmetic — `maxhp * (1/3)` is not `maxhp / 3`, and a body at exactly one third would be refused a boost it is owed. An `onlyWhen` the engine still cannot read returns null, still refuses, and is now counted. The narrowed shape's membership went 5 → 9 and `tests/test-damage-stages.js` still reads 1728/1728 exact with 0 at the wrong stage. 9,141 uses, on today's corpus.

ROADMAP #88 AND #91 — ONE PIN WAS ONE CORNER, AND A CLICK WAS COUNTED AS A TEST (3.73.0). Every die in the differential was pinned a single way, which bought determinism — any difference is a bug, no statistics — and paid for it in coverage nobody had priced. The speed tie always resolved the same direction, every move below 100 accuracy MISSED ON BOTH SIDES, and damage was always the maximum roll, which is the one roll where the crit's wrong position happened to come out right. Rock Slide had never connected in this instrument; under the new arms it misses in one and hits in another, and a crit lands in the bottom arm and not the top. The pin set is now a declared run parameter, digested into `mode`, and a before/after pair whose pins differ is REFUSED rather than reported. Separately, coverage credit moved from the CLICK to the OBSERVED EFFECT: the old rule incremented when an entity was clicked and never asked whether the move did anything, so Haze clicked into a board with no boosts on it — a no-op — marked Haze exercised and stopped the steering selecting it. Five rows were clicked-or-present and did nothing at all: `critDamageUp`, `preventsSwitch`, `privateWeather`, `clearsScreens` and `preTurnShield`. The old rule called all five covered. **THE BASELINE IS RESET: both changes alter which games get played, so no run after this is comparable with the turn-1 figure published at 3.71.0 or with** `data/state-ladder.json`. And an ENGINE defect fell out of the tie work, filed rather than fixed here: the two engines have disagreed about EVERY speed tie for the life of this instrument — the authority resolves a tie to the LATER body in input order, `sortTurnOrder` draws one tie value per action from a constant scalar so the sort is stable and takes the EARLIER one. The instrument's own header claimed the pin made them agree by construction; that claim was false and was repeated as fact before it was checked. `sortTurnOrder` is the live engine, not instrument code.

3.82.0 — THE FIRST ENGINE FIX OF THE QUEUE LANDS: THE VOLATILE DURATION FAMILY, 9,092 USES, AND IT WAS THE PERISH SONG BUG A SECOND TIME. Showdown decrements a volatile's duration inside the Residual event, so one applied on turn N has already spent a turn by the end of it. That defect was documented in this engine for Perish Song, fixed for Perish Song, and left standing for every other duration-bearing volatile. Taunt and Disable now match the official engine; Encore's counter row is gone and only a separate HP row remains. Whole-game board agreement rose 76.9% to 78.9% on a paired differential, the roster's moves queue fell from 52 to 50, and the census did not move. The previously published baseline could not be reproduced because the census digest and the team store had both shifted underneath it — so the run was discarded and re-taken paired. The delta is the measurement.

3.81.0 — THE QUARANTINE REACHED THE BOARD, AND THE CHECKER THAT POLICES IT WAS BLIND TO THE PUBLISHED COPY. Thirteen slots on ABRA WORLD's status board now render as a redaction bar rather than a number, each carrying the artifact, the reason and the command that re-runs it. Two defects in the guard itself were found doing it: its own selftest had gone red — an `all`-stage artifact was matching ANY requested stage name, so "a missing stage must FAIL" stopped being enforced — and its citation walker looked at `docs/` and `web/` but never at `app/`, which is the copy a visitor actually loads. Five withheld verdicts were being published from `app/` the whole time the check read green.

3.80.0 — THE DELIBERATE ROSTER'S INERT BUCKET COLLAPSED BY 94.1% OF ITS USAGE, AND A FAMILY OF ABILITIES WORTH 8,524 USES TURNS OUT NEVER TO HAVE FIRED. 124 abilities were falling through a catch-all that stages a plain attack, so the condition each one needs was never created and the roster honestly reported INERT — which reads as "nothing to test" when the truth is "never tested". Fifteen new shape rules take that bucket to 59 abilities / 4,261 uses, all 22 ability rules caught their own break, and nothing left in the bucket is above 500 uses. The first real defect it found: Blaze, Torrent, Overgrow and Swarm carry their below-1/3-HP condition as PROSE, and the consumer refuses any condition it cannot evaluate — so the pinch family has never once fired. The roster's artifacts are now written per stage, so the quarantine gate reads measurement rather than absence.

3.79.0 — EVERY FIGURE DOWNSTREAM OF MEDICHAM IS NOW WITHHELD RATHER THAN CAPTIONED, AND THE DELIBERATE ROSTER'S MOVES STAGE RAN FOR THE FIRST TIME. Will's standing call: *"all engines that take medicham's output should be regarded as out of date and we should stop referencing them until medicham is up to date and we can rerun them."* 34 of 114 artifacts are downstream and are no longer printed at all — R1, R2, R3, R4, leaf calibration and the weights among them — because a caption is not a quarantine: `PRE-CHANGE` had been printed beside those numbers for days and they went on being quoted anyway. Membership is derived from the dependency graph, not typed, and the gate that lifts it is computed from the differential and the roster, where a MISSING stage counts as failing. The roster gained 26 move shape rules and staged all 500 legal moves for the first time, returning a 79-row queue over ~15,000 uses. Its control arm was found to be measuring the CONTROL rather than the subject, which made six ability findings false; **Weather Ball, Sand Rush and Damp are retracted as defects and are correct.**

3.78.0 — THE SHEET'S REAL NATURE NOW REACHES BOTH ENGINES, AND THE TURN-1 NUMBER FELL. THAT IS THE INSTRUMENT GETTING HONEST. The whole-game differential built every body `Serious` while the stored sheet beside it said `Modest`, and with every body flat AND Serious, 326 of 357 species in the format share a Speed with some other species — so the rig MANUFACTURED speed ties and almost never tested a real speed differential. Carrying the declared nature cut the tied groups the resolver has to break from 348,595 to 243,467 over the same 1,998 games, a 30.2% fall. The instrument's own numbers went DOWN, as predicted before the run: the board at the end of turn 1 is identical in 97.4% of games flat against 97.3% natured, games whose board never parted 80.8% against 78.8%, and the median turn of first board divergence one turn earlier at 7. Encore divergences nearly doubled (10 to 19 games), which is what a duration volatile that only bites when turn order does looks like when turn order starts being tested. THE SPREADS REMAIN ABSENT AND ALWAYS WILL BE — a Showdown open team sheet does not reveal them (`"evs": null` on 173,784 of 173,784 stored bodies), so this narrows the declared gap and does not close it. Neither engine is told the other's answer: both are told the nature and each computes, and the alignment assertion still reads 0. It read 21 on the first run and all 21 were Ditto — entry-time Imposter had already transformed the medicham body, and the harness was writing the copied stat line onto Showdown's Ditto before the game began. Census 324 live, 0 missing, unchanged.

3.77.0 — CONFUSION DID NOT EXIST, AND BURN HAD NEVER BEEN ON A BOARD. The confusion volatile was written and never read or ticked, so Hurricane's secondary — 3,779 uses — fell through every branch, and the two berries that clear confusion looked dead because there was nothing to clear. The sleep counter was an ordering bug: the authority runs sleep before flinch, ours ran flinch first, so a body that was asleep AND flinched never ticked and woke a full turn late. Burn, by contrast, is CORRECT and was confirmed rather than changed — but it had never once been staged, because Will-O-Wisp is 85-accurate and the harness pin makes every sub-100 move miss. The freeze timer is correct too; what was missing was the instrument, which carried no freeze counter at all, so the engine's value could drift and no measurement would see it. Census 324 live, 0 missing.

3.77.0 — THE ACROSS-A-SWITCH ARM FOUND A DEFECT ONE DAY OLD THAT FIXING SOMETHING ELSE CREATED. A transform never reverts when the body leaves the field: the authority clears it in `clearVolatile`, and this engine sets the flag and never unsets it. Since the transform also overwrites the body's name, stats, types, moves, boosts and ability, a benched Ditto is PERMANENTLY the thing it copied — so the two engines then choose replacements from benches that no longer describe the same Pokemon, and worse, a Ditto can only ever transform ONCE PER BATTLE, because the guard refuses a second. Re-copying is the entire function of the Pokemon. Imposter first fired the day before, and the out-and-back scenario that exposes this only became expressible hours earlier. The roster's two owed arms — across a switch, and at the exact HP line — are both built, both red-demonstrated, and Speed Boost and Focus Sash both match.

3.77.0 — A STAGED SCENARIO CAN NOW SWITCH, SO A MID-TURN ENTRANT IS EXPRESSIBLE FOR THE FIRST TIME. The scenario driver understood only a move; every other step became a pass, so no staged test could put a body on the field part-way through a turn. That single gap blocked three things at once: Speed Boost's entry gate, which exists only for a body that just switched in; Hunger Switch's flip and Zero to Hero's switch-out transform; and the whole across-a-switch arm of the roster. Four of the six engine defects found the day before were about a MOMENT rather than an effect, and no scenario without an entrant can express one. Verified end to end: Espathra switches in and reads +0 Speed in both engines on the turn it arrives, then +1 at the end of the next, with all 131 fields identical on both boundaries.

3.77.0 — ALL 316 ABILITIES STAGED DELIBERATELY, AND A FREE +6 ATTACK FELL OUT. Anger Point and Justified are one defect twice: a conditional boost-on-being-hit whose condition is never checked, so Anger Point grants +6 Attack off an ordinary hit where it requires a crit, and Justified grants +1 off a Poison move where it requires Dark. Hustle applies no 1.5x Attack at all. Two facts about the instrument matter as much: Gastro Acid does not suppress an ability here, and since suppression is the ONLY control available to 23 abilities, checking that control against a known-live fixture is what stopped five more from being published as dead for the control's failure rather than their own. And a fact about the regulation rather than the simulator: 113 of 316 legal abilities have NO legal carrier, so the effective roster of this format is about 203.

3.77.0 — FIVE MECHANICS THAT DID NOTHING, AND ONE THAT WAS ALREADY RIGHT. Imposter never transformed Ditto; Hunger Switch never flipped Morpeko; Knock Off took its 1.5x against an item it cannot remove; Fling never became an attack at all, because a base power of 0 made the click fail a `hasPower()` gate; and Roar's phaze branch held a Pokemon-first target, so a phaze after a pivot dragged nobody — the SIXTH site missed by the slot-first sweep, and at priority -6 the worst possible place to hold a body rather than a slot. Mawile's mega ability swap, which had been blamed for a whole family of Attack-stage divergences, WAS ALREADY CORRECT: the scenario was board-identical on its first run, and deleting the swap deliberately parts two fields at once, so the symptoms were real symptoms of a bug this engine does not have. Census 319/319 live, 0 missing; the staged harness now carries 24 scenarios, all clean and all breakable.

3.77.0 — THE INSTRUMENT RESOLVED A SWITCH BY TWO DIFFERENT KEYS AND FAILED SILENTLY BOTH WAYS. The driver names a bench member by Showdown's species id; the Showdown side looked it up by species id and the medicham side by the body's DISPLAY NAME. Those agree until a body is renamed — which this engine began doing the day before, when Disguise started renaming a busted Mimikyu, Zero to Hero started renaming Palafin, and Hunger Switch was queued to flip Morpeko every turn. After a rename the two keys part and that body can never be switched to again. Neither side raised anything: an unresolved lookup answered `pass` on both, so one engine could switch while the other stood still, producing a different board with no evidence attached. The key is now stamped at build time from the same expression the driver uses, and a miss is counted and printed beside the other declared gaps (0/0 over 120 games). This is an INSTRUMENT change rather than an engine one, so it alters what a measurement sees; it was also LATENT UNTIL THE FORME FIXES LANDED, and the deliberate-roster build would have walked into it.

NO STAGE MOVED IN 3.77.0, AND THE VERSION MOVED ANYWAY — the reason is worth stating rather than pinning. WIRE 139 changed WHICH BODY a move resolves against (the slot, not the Pokemon, which is what `Battle#getTarget` does), and WIRE 140 added Ally Switch, which moves two bodies between slots mid-turn. Neither touches a multiplier or its stage, so every row in the table below still holds exactly as measured — but both sit UPSTREAM of the whole table: a multiplier applied at the right stage to the wrong defender is wrong for a reason this document cannot see. Said here so a later reader does not conclude the audit was re-run.

WIRE 133–138 ADDED ONE MULTIPLIER TO THIS AUDIT AND SETTLED THE SPEED-TIE PARAGRAPH ABOVE (3.74.0). The paragraph at the head of this file was RIGHT that the two engines disagreed about every speed tie and RIGHT that `sortTurnOrder` is the live engine, and its DIAGNOSIS was incomplete: "the authority resolves a tie to the LATER body in input order" is what the authority PRODUCES UNDER THIS HARNESS'S PIN, which replaces `PRNG.shuffle` with a no-op — it is not a rule. The rule is `Battle#speedSort`, a SELECTION SORT whose swaps move UNTIED elements around, ending in a

Fisher-Yates over the tied group: a speed tie is a COIN FLIP. The engine now performs the same selection sort and resolves the residual group with the per-action uniform key it already drew, so both engines land on the same body under identical pinned dice and both are a fair coin under real ones. `tests/test-speed-tie.js` proves it in both team orientations, with the tied pair on one side, on a three-way tie, and against a no-tie control. **The one change to the DAMAGE formula itself is WONDER ROOM** (`swapsDefences` , 11 uses), which had no consumer at all. It swaps the STORED DEFENSIVE STAT and NOT the boost stage — `Pokemon#getStat` swaps `storedStats` at the top of the function and then applies `boosts[statName]` , the ORIGINAL stat's stage — so the swap is applied to `ⓓ` before the stage multiplier and `_dKey` is deliberately left unrewritten. It rides on whichever defence the move attacks into, so Psyshock and Body Press inherit it through `statSwap` rather than through a second rule. Nothing else in §2a or §3 moved.

THIS AUDIT HAS BEEN LANDED. ROADMAP #92, 3.73.0. Everything §6 lists as open work is fixed, and the numbers below are now HISTORY — they describe the engine at release `dc3c43336539` , not the engine in the tree. **Read §2a and §3 as the record of what was wrong, not as a description of what is wrong.** What replaced them: - every `onBasePower` member is folded into ONE relay spent once, and every `onModifyAtk` / `onModifySpA` / `onModifyDef` / `onModifySpD` member into two more — the STAGE and the CHAIN halves of §2's finding, which had to move together; - Friend Guard is inside the `ModifyDamage` chain rather than beside it; Helping Hand and the ally multiplier reach `dmgRange` on a seventh argument, because what it cannot DERIVE it can be TOLD; - the rolled crit's $\times 1.5$ is applied inside `dmgRange` before the randomizer, where the authority applies it, and Sniper has left that multiply for the final chain; - the four field terrains exist for the first time, with the authority's own grounded subject. **The claim is checked rather than asserted:** `tests/test-damage-stages.js` runs 54 scenarios $\times$ 16 damage rolls $\times$ 2 crit states against Showdown's own `moveHit` and demands EXACT equality — **1,728/1,728** — and it was shown RED on two deliberate reversions before being trusted. The census carries five of these as probes (`move|setsTerrain` $\times$ 4, `ability|damageBoost`), which is what the last paragraph of this header said it did not yet do. **§2c is the part that keeps its original force:** it is the list of things that were checked and found CORRECT, and the gate now re-checks every one of them on every run. Four things are still not fixed and each is named with its reason in `docs/ENGINE.md` — Charge (no volatile exists to read), `terrainScaled` 's grounded SUBJECT (the tag carries none), Rivalry (no gender in `MC.mons`), and the artifact storing 1.3 as a float where the authority spells `[5325,4096]` (the engine carries a four-entry override that the gate re-derives from the live dex).

Audited against engine release `dc3c43336539` and the Showdown checkout at `SHOWDOWN_PATH` . Every rate in this document was measured in the session that wrote it.

Will, 2026-08-07: "LETS CHECK THE DAMAGE FORMULA FOR ALL ITS COMPONENTS AND COMPARE OURS AGAINST SHOWDOWN."

Read against the frozen release `dc3c43336539` , not the live tree — `engine/medicham2-browser.js` was being edited by another agent while this was written. Every line number in the "ours" column is a line in `data/releases/dc3c43336539/engine/medicham2-browser.js` . Every line number in the authority column is `pokemon-showdown/sim/battle-actions.ts` or `sim/battle.ts` .

This document is an AUDIT. It changes no engine file and lands no mechanic. Nothing here is a probe in `tests/test-mechanics.js` yet, so nothing here is carried by the census — that is the next pass's job.
